# PYTHON AUTOMATION FOR BEGINNERS

25 Practical Projects for Files, Spreadsheets, Reports & Everyday Work



PYTHON

# AUTOMATION FOR BEGINNERS

25 Practical Projects for Files, Spreadsheets, Reports & Everyday Work

Edition 1 · 2026

## Copyright and notice

© 2026. All rights reserved.

You may copy the templates, worksheets, scripts and checklists in this book for your own personal or business use. You may not resell, share or distribute the book itself. Brief quotations are permitted with credit.

## Educational notice

Code is provided as is, for learning. Run scripts on copies of your files first and keep backups. Each project states what was and was not tested.

## Sources

Statements of fact are supported by the references at the end of the book, with the date each source was checked. Web pages change; confirm current details before relying on them.

# Contents

**Introduction** 6

**Chapter 1 Introduction to Python automation** 9

**Chapter 2 Installing Python and setting up** 11

**Chapter 3 Python foundations needed** 14

**Chapter 4 Files, folders and batch renaming** 18

Project 1. Bulk file renaming 19
Project 2. Organising a folder by file type 23
Project 3. Duplicate file detection 27
Project 4. Automated file backup with timestamps 31
Project 5. Folder size report 35
Project 6. Image file organisation by date 38

**Chapter 5 CSV processing** 42

Project 7. CSV cleanup 43
Project 8. Splitting a large CSV 47
Project 9. Simple expense tracking 50
Project 10. Contact list de-duplication 54
Project 11. Data validation report 58
Project 12. Invoice number generator 62

**Chapter 6 Excel automation** 66

Project 13. Excel report generation 67
Project 14. Merging spreadsheets 71

**Chapter 7 PDF and document processing** 75

Project 15. PDF text extraction 76
Project 16. Merging PDF files 80
Project 17. Splitting a PDF by page range 83

**Chapter 8 Text processing** 86
Project 18. Word frequency counter 87
Project 19. Find and replace across files 90
Project 20. Release notes text cleaner 94
Project 21. Log file summary 97

**Chapter 9 Reports and spreadsheets** 100
Project 22. Monthly report generation 101
Project 23. Email preparation drafts 106

**Chapter 10 APIs and responsible web data usage** 110
Project 24. API data retrieval 112

**Chapter 11 Scheduling** 116
Project 25. Scheduled report preparation 119

**Chapter 12 Debugging, security and next steps** 123

**Conclusion** 126

**Appendix A. Python cheat sheet** 127

**Appendix B. Common error reference** 128

**Appendix C. Glossary** 130

**Appendix D. Project checklist** 132

**Appendix E. 30-day learning plan** 133

**Appendix F. References and further reading** 135

# Introduction

Most office and study work contains tasks that repeat: renaming forty files, tidying a spreadsheet export, joining PDFs, writing the same monthly summary. This book teaches you to hand those tasks to short Python scripts. It assumes you can use a computer comfortably and have never programmed, or have only dabbled.

You get 25 complete projects. Each starts from a problem, gives a full script, explains the important lines, shows the real output I got when I ran it, and lists the errors you are likely to meet. The scripts are small on purpose: one script, one job, readable in a few minutes.

## What is tested and what is not

All 25 scripts were run on 2026-11-07 on Windows 11 with Python 3.14.4, inside a virtual environment, with openpyxl 3.1.5 and pypdf 7.0.0. Every "Sample output" in this book is the real output of that run. The data is invented sample data, included in the optional download.

Not tested: macOS, Linux, any scheduler (Windows Task Scheduler, cron, launchd), any mail program opening the draft emails, OCR for scanned PDFs, and Excel itself opening the generated workbooks. The scripts use `pathlib` and standard libraries so they should work on other systems, but I have not run them there. Where a platform detail differs, the book says so. Outputs on macOS or Linux will show `/` instead of `\` in paths.

A smaller compatibility check was also run on Python 3.12.12 (Windows 11, openpyxl 3.1.5, pypdf 4.3.1, 2026-11-07): all 25 scripts compiled and showed their help text, and nine representative scripts (Projects 3, 7, 9, 13, 14, 15, 18, 21 and 22) gave the same results as on 3.14.4. The other scripts were not run in full on 3.12, and Python 3.13 was not tested.

The one network project (Project 24) reached its public test API when I ran it. Its failure path was also run, using an address that refuses connections.

## Which Python version

Use Python 3.12 or newer. At the time of writing python.org offers newer series than 3.14 as well; download the current stable release from python.org and check the documentation for your version at docs.python.org. The scripts avoid new features and should work on any 3.12 or newer, but the full test run used 3.14.4 and a smaller check used 3.12.12.

## How to use this book

- Read Chapters 1 to 3 once. If you already program, skim Chapter 3.
- Do the projects in order within a chapter, or jump to the one you need. Each project stands alone.
- Run every script first on the sample data, then on a copy of your own files, never on the only copy.
- Every project has the same twelve labelled parts, so you can find the code, the commands and the warnings quickly.

## Optional bonus download

The folder `source`, with all 25 scripts, the sample data, `requirements.txt` and a README, is available as `Python_Automation_Project_Source.zip`. It is optional: every script is

printed in full in this book, but typing well over a thousand lines of code is not a good use of your time. The one script not printed with a project is `make_sample_data.py`, which creates the sample data.

## What this book does not do

It does not teach all of Python, web scraping, data science or machine learning. It does not promise that automation saves a particular amount of time; that depends on how often your task repeats and how long the script takes you to adapt. It does not replace professional advice for legal, tax or privacy questions about the data you process.

## How this book was researched

On 2026-11-07 I reviewed competing Python automation titles on Gumroad (several pages returned only a title, so I could not compare contents), read the official documentation for pypdf, `venv`, JSONPlaceholder, robots.txt (RFC 9309) and Windows Task Scheduler, and searched for reader complaints. The pattern I found, with limited evidence, was: books that are Excel-only or a bag of unrelated scripts; code without tested output; and weak coverage of safety. This book combines files, spreadsheets, PDFs, text, reports and scheduling for beginners, shows tested output, and makes dry runs and backups part of every script that changes something. Sources are listed in Appendix F.

CHAPTER 1

# Introduction to Python automation

## What automation means here

Automation means writing down the steps of a task once, precisely, so a computer can repeat them. A script is a text file of such steps in Python. You run it, it does the work, and it can report what it did.

Python suits this work for three reasons. Its syntax is close to plain English. Its standard library already handles files, CSV, dates, regular expressions, email messages and web requests. And a large set of free packages covers formats the standard library does not, such as Excel files and PDFs.

## Is a task worth automating

Use this quick test before you write any code.

- [ ] The task repeats, or it is large (hundreds of files or rows).
- [ ] The steps are the same every time, or the differences can be described as rules.
- [ ] A mistake is cheap to undo, or you can keep a backup.
- [ ] You can describe the correct result in a sentence.

If the first box is not ticked, doing it by hand may be faster. A simple estimate helps: if the task takes 10 minutes and happens weekly, that is about 520 minutes a year (10 x 52). If writing and testing the script

takes three hours (180 minutes), it pays back in a little over a third of a year. Your numbers will differ; the arithmetic is the point.

## The shape of every script in this book

Each project follows the same pattern, which is also a good habit for your own scripts:

1. Read the arguments (which folder, which file).
2. Validate them and stop with a clear message if something is wrong.
3. Work out what would be done, before doing it.
4. Print the plan. If the script changes things, stop here unless `--apply` was given.
5. Do the work without overwriting anything.
6. Report what happened.

## Tasks this book covers

| Area | Chapter | Projects |
|---|---|---|
| Files and folders | 4 | 1 to 6 |
| CSV data | 5 | 7 to 12 |
| Excel | 6 | 13 to 14 |
| PDF | 7 | 15 to 17 |
| Text | 8 | 18 to 21 |
| Reports and drafts | 9 | 22 to 23 |
| APIs | 10 | 24 |
| Scheduling | 11 | 25 |

CHAPTER 2

# Installing Python and setting up

## Install Python

Download the current stable Python from the official page, https://www.python.org/downloads/, and follow the installer for your system. Python 3.12 or newer is required.

- **Windows:** run the installer. If it offers "Add python.exe to PATH", tick it. The installer also provides the `py` launcher. Check with `py --version` in PowerShell or Command Prompt.
- **macOS:** use the installer from python.org. Check with `python3 --version` in Terminal.
- **Linux:** Python 3 is usually installed. Check with `python3 --version`. On Debian or Ubuntu you may need `sudo apt install python3-venv python3-pip`. This is the general pattern on those systems; I did not test it.

On Windows, type `py` where this book shows `py`. On macOS and Linux, replace `py` with `python3` in every command. Both of these are the same program: `py script.py` on Windows and `python3 script.py` elsewhere.

If your computer has several Pythons (for example one bundled inside another application), check which one runs: `py -0p` on Windows lists the installed ones. Use a normal Python installation, not the copy inside another program.

## Virtual environments

A virtual environment is a private folder of packages for one project. It stops one project's packages from clashing with another's and keeps your main Python clean. Create one inside the folder where you keep the book's files:

```
py -m venv .venv
```

Activate it. The commands below are from the official `venv` documentation:

```
# Windows PowerShell
.venv\Scripts\Activate.ps1

# Windows Command Prompt
.venv\Scripts\activate.bat

# macOS and Linux (bash or zsh)
source .venv/bin/activate
```

On PowerShell you may first need `Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser`, which the same documentation mentions. The prompt shows `(.venv)` when active. Type `deactivate` to leave. Once active, `python` points to the environment's Python.

## Installing packages with pip

Always call pip through Python so you are sure which Python receives the package:

```
py -m pip install openpyxl pypdf
py -m pip list
```

The bonus download has a `requirements.txt` so one command installs both:

```
py -m pip install -r requirements.txt
```

Only 6 of the 25 projects need a package: openpyxl (Projects 13, 14, 22) and pypdf (Projects 15, 16, 17). The helper `make_sample_data.py` also needs openpyxl. The other 19 projects use the standard library only.

## Running a script

Save the script as a `.py` file, open a terminal in its folder, and run:

```
py project05_folder_size_report.py sample_data/project_files
```

Anything after the file name is an argument the script can read. If you see `can't open file`, you are in the wrong folder; use `cd` to move, or give the full path. Forward slashes work in paths on Windows too.

## Choosing an editor

Any plain text editor works. Beginners usually do better with an editor that colours code and shows errors: Visual Studio Code, PyCharm Community Edition, or IDLE, which comes with Python. Avoid word processors; they insert curly quotes that break code. Save files as UTF-8.

## Setup checklist

- [ ] `py --version` (or `python3 --version`) shows 3.12 or newer.
- [ ] A virtual environment is created and active.
- [ ] `py -m pip install openpyxl pypdf` worked.
- [ ] You unpacked the bonus download, or created a folder for your own scripts.
- [ ] In that folder, `py make_sample_data.py` printed "Sample data written to ...".

CHAPTER 3

# Python foundations needed

This chapter covers only what the projects use. Type the examples into a file and run them, or into the interactive prompt (`py` with no arguments; `exit()` to leave).

## Variables and types

```
name = "Ada"          # str: text
count = 3             # int: whole number
price = 4.5           # float: decimal number
paid = True           # bool: True or False
print(name, count * price, paid)
```

For money use `Decimal` (Project 9), because floats store some decimals approximately.

## Strings and f-strings

```
file = "  Report_Final.TXT "
print(file.strip().lower())      # report_final.txt
print(file.split("_"))           # ['  Report', 'Final.TXT ']
number = 7
print(f"invoice_{number:03d}.pdf")   # invoice_007.pdf
```

An f-string starts with `f` and puts values inside `{}`. After the colon you can format: `03d` pads to three digits, `.2f` gives two decimals.

## Lists and dictionaries

```
files = ["a.txt", "b.csv"]
files.append("c.pdf")
for item in files:
    print(item)

totals = {"North": 10, "South": 4}
totals["East"] = 7
print(totals["North"], totals.get("West", 0))   # 10 0
```

A list keeps order; a dictionary looks values up by key. `.get(key, default)` avoids an error when the key is missing.

## Decisions and loops

```
for n in range(1, 6):
    if n % 2 == 0:
        print(n, "even")
    else:
        print(n, "odd")
```

Indentation (four spaces) is how Python knows what belongs to the `for` or `if`. A missing or uneven indent gives `IndentationError`.

## Functions

```
def vat_total(net, rate=0.2):
    """Return net plus tax."""
    return net * (1 + rate)

print(vat_total(100))        # 120.0
```

A function names a piece of work. Small functions are easier to test and reuse, which is why the projects split work into them.

## Paths with pathlib

```
from pathlib import Path

folder = Path("sample_data")
for path in folder.iterdir():
    print(path.name, path.suffix, path.is_file())

target = folder / "notes" / "a.txt"      # the / joins parts
print(target.exists())
```

`pathlib` builds paths that work on any operating system. Useful pieces: `.name`, `.stem` (name without extension), `.suffix`, `.parent`, `.exists()`, `.is_file()`, `.glob("*.csv")`, `.rglob("*")` (include sub-folders).

## Reading and writing text

```
path = Path("hello.txt")
path.write_text("Hello\nWorld\n", encoding="utf-8")
text = path.read_text(encoding="utf-8")
print(text.splitlines())

with path.open("a", encoding="utf-8") as fh:   # "a" appends
    fh.write("Third line\n")
```

Always state `encoding="utf-8"`. Use `with` so the file closes by itself. Mode `"w"` replaces a file's content, which is why the projects check `exists()` before writing.

## Exceptions

```
try:
    value = int("abc")
except ValueError as error:
    print("Not a number:", error)
```

Catch the specific error you expect. Catching everything hides bugs; Project 25 does it only at the top of a scheduled job, and logs the full traceback.

## The main guard and arguments

```
import argparse

def main(argv=None):
    parser = argparse.ArgumentParser(description="Demo")
    parser.add_argument("folder")
    parser.add_argument("--apply", action="store_true")
    args = parser.parse_args(argv)
    print(args.folder, args.apply)
    return 0

if __name__ == "__main__":
    raise SystemExit(main())
```

`if __name__ == "__main__":` runs `main()` only when you run the file directly, not when another script imports it. `argparse` reads the command line and creates `--help` for free. The returned number becomes the exit code: 0 means success, anything else means a problem, which schedulers can detect.

## Foundations checklist

- [ ] I can explain the difference between a list and a dictionary.
- [ ] I know why `"w"` mode needs care.
- [ ] I can run a script with an argument.
- [ ] I can read the last line of a traceback (Chapter 12).

CHAPTER 4

# Files, folders and batch renaming

File work is the safest place to start: the results are visible in your file manager, and the dangerous operations (rename, move, delete) are exactly where good habits matter most. Three rules run through the six projects.

- **Dry run first.** The script prints what it would do. A flag such as `--apply` turns on the real action.
- **Never overwrite.** If a target exists, the script stops, skips, or chooses a new name.
- **Never delete.** No project in this book deletes a file. Where deletion might be wanted, the script reports and you decide.

## Project 1. Bulk file renaming

**Problem:** A folder holds files named IMG_4471.jpg, Beach day.jpg and notes final v2.txt. You want one consistent pattern such as trip_001.jpg.

**Expected result:** Every file renamed to prefix_NNN.ext in sorted order, lowercase extension, plus an undo log. Without --apply nothing changes.

**Required tools:** Python standard library only.

**Installation:** nothing to install. Use `py` on Windows or `python3` on macOS and Linux to run it.

**Complete source code** (`project01_bulk_rename.py`):

```
"""Project 1: rename the files in one folder to prefix_001.ext,
prefix_002.ext.
Dry run by default; add --apply to really rename."""
import argparse
import csv
from pathlib import Path


def plan_renames(folder, prefix, start):
    # Sort so the numbering is the same every time you run the
script.
    files = sorted(p for p in folder.iterdir()
                   if p.is_file() and p.name !=
"rename_undo_log.csv")
    plan = []
    for number, path in enumerate(files, start=start):
        new_name = f"{prefix}_{number:03d}{path.suffix.lower()}"
        plan.append((path, path.with_name(new_name)))
    return plan


def main(argv=None):
    parser = argparse.ArgumentParser(description="Bulk rename
files.")
    parser.add_argument("folder", type=Path)
    parser.add_argument("--prefix", default="file")
```

```
    parser.add_argument("--start", type=int, default=1)
    parser.add_argument("--apply", action="store_true",
                        help="really rename (default is a dry
run)")
    args = parser.parse_args(argv)

    if not args.folder.is_dir():
        print(f"Error: {args.folder} is not a folder.")
        return 1
    plan = plan_renames(args.folder, args.prefix, args.start)
    # Refuse to start if a target name already belongs to a
different file.
    sources = {src for src, _ in plan}
    for src, dst in plan:
        if dst.exists() and dst not in sources:
            print(f"Stopped: {dst.name} already exists and is not
in the plan.")
            return 1
    if len({dst for _, dst in plan}) != len(plan):
        print("Stopped: two files would get the same new name.")
        return 1

    for src, dst in plan:
        print(f"{src.name}  ->  {dst.name}")
    if not args.apply:
        print(f"Dry run: {len(plan)} files would be renamed. Add
--apply.")
        return 0

    # Two steps (temporary names first) so a swap such as a->b,
b->a is safe.
    temp = []
    for src, dst in plan:
        tmp = src.with_name(src.name + ".renaming")
        src.rename(tmp)
        temp.append((tmp, dst, src))
    for tmp, dst, _ in temp:
        tmp.rename(dst)
    # Undo log: lets you reverse the rename by hand if you change
your mind.
    log = args.folder / "rename_undo_log.csv"
    with log.open("w", newline="", encoding="utf-8") as fh:
        writer = csv.writer(fh)
        writer.writerow(["old_name", "new_name"])
        writer.writerows((s.name, d.name) for _, d, s in temp)
    print(f"Renamed {len(plan)} files. Undo log: {log.name}")
```

```
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

**Code explanation:**

- `plan_renames` builds the whole list of (old, new) pairs before touching anything. That is what makes a dry run possible.
- `sorted(...)` fixes the order, so the numbering is repeatable. `f"{number:03d}"` pads 1 to 001.
- The two checks stop the script if a target name already belongs to another file, or if two files would get the same name.
- Renaming goes through a temporary `.renaming` name first. Without that step, a rename such as a to b while b to a exists would overwrite a file.
- The undo log is a CSV of old and new names, so you can reverse the job by hand.

**Sample input:** Folder `sample_data/rename_demo` with five files: `Beach day.jpg`, `budget (copy).txt`, `IMG_4471.jpg`, `IMG_4472.jpg`, `notes final v2.txt`.

**Sample output** (real output from my test run, Windows 11, Python 3.14.4, 2026-11-07):

Dry run (default):

```
Beach day.jpg  ->  trip_001.jpg
budget (copy).txt  ->  trip_002.txt
IMG_4471.jpg  ->  trip_003.jpg
IMG_4472.jpg  ->  trip_004.jpg
notes final v2.txt  ->  trip_005.txt
Dry run: 5 files would be renamed. Add --apply.
```

Real run:

```
Beach day.jpg  ->  trip_001.jpg
budget (copy).txt  ->  trip_002.txt
IMG_4471.jpg  ->  trip_003.jpg
IMG_4472.jpg  ->  trip_004.jpg
notes final v2.txt  ->  trip_005.txt
Renamed 5 files. Undo log: rename_undo_log.csv
```

**How to run it** (from the `source` folder):

```
py project01_bulk_rename.py sample_data/rename_demo --prefix trip
py project01_bulk_rename.py sample_data/rename_demo --prefix trip
--apply
```

These commands change or create files. To repeat a test, delete the `sample_data` folder and run `py make_sample_data.py` again.

**Common errors and fixes:**

- **Stopped: trip_001.jpg already exists and is not in the plan.** A file with a target name is already there. Choose another --prefix or --start.
- **PermissionError.** A file is open in another program. Close it and run again.

**Safe-use notes:** Run the dry run first and read every line. Keep the undo log. The script renames only files directly in the folder, never sub-folders.

**Optional challenge:** Add an option --undo that reads rename_undo_log.csv and restores the old names.

## Project 2. Organising a folder by file type

**Problem:** Your Downloads folder is a heap. You want Images, Documents, Spreadsheets, Archives, Audio and Other folders.

**Expected result:** Each file moved into a category folder chosen by its extension. Name clashes get _1, _2 added. Dry run by default.

**Required tools:** Python standard library only.

**Installation:** nothing to install. Use `py` on Windows or `python3` on macOS and Linux to run it.

**Complete source code** (`project02_organise_by_type.py`):

```
"""Project 2: move the files in a folder into sub-folders by file
type.
Dry run by default; add --apply to really move files."""
import argparse
import shutil
from pathlib import Path

CATEGORIES = {
    "Images": {".jpg", ".jpeg", ".png", ".gif", ".webp"},
    "Documents": {".pdf", ".doc", ".docx", ".txt", ".md"},
    "Spreadsheets": {".xlsx", ".xls", ".csv"},
    "Archives": {".zip", ".7z", ".tar", ".gz"},
    "Audio": {".mp3", ".wav", ".m4a"},
}


def category_for(path):
    suffix = path.suffix.lower()
    for name, extensions in CATEGORIES.items():
        if suffix in extensions:
            return name
    return "Other"


def free_name(target):
    """Return target, or target with _1, _2 ... so nothing is
overwritten."""
```

```
    counter = 1
    candidate = target
    while candidate.exists():
        candidate = target.with_name(f"{target.stem}_{counter}
{target.suffix}")
        counter += 1
    return candidate


def main(argv=None):
    parser = argparse.ArgumentParser(description="Organise files
by type.")
    parser.add_argument("folder", type=Path)
    parser.add_argument("--apply", action="store_true")
    args = parser.parse_args(argv)
    if not args.folder.is_dir():
        print(f"Error: {args.folder} is not a folder.")
        return 1

    counts = {}
    # Only files directly inside the folder; sub-folders are left
alone.
    for path in sorted(p for p in args.folder.iterdir() if
p.is_file()):
        category = category_for(path)
        target = free_name(args.folder / category / path.name)
        counts[category] = counts.get(category, 0) + 1
        print(f"{path.name}  ->  {category}/{target.name}")
        if args.apply:
            target.parent.mkdir(exist_ok=True)
            shutil.move(str(path), str(target))
    summary = ", ".join(f"{k}: {v}" for k, v in
sorted(counts.items()))
    mode = "Moved" if args.apply else "Dry run, would move"
    print(f"{mode} {sum(counts.values())} files ({summary}).")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

**Code explanation:**

- `CATEGORIES` is a dictionary from folder name to a set of extensions. Editing this one table customises the script.
- `category_for` lowercases the suffix so `.JPG` and `.jpg` behave the same.
- `free_name` loops until it finds a name that does not exist. This is how the script avoids overwriting.
- `shutil.move` is only called when `--apply` is present.

**Sample input:** Folder `sample_data/downloads` with 12 files such as `holiday.jpg`, `cv_2026.pdf`, `budget.xlsx`, `setup.exe` and `readme` (no extension).

**Sample output** (real output from my test run, Windows 11, Python 3.14.4, 2026-11-07):

Dry run:

```
backup.zip  ->  Archives/backup.zip
budget.xlsx  ->  Spreadsheets/budget.xlsx
cv_2026.pdf  ->  Documents/cv_2026.pdf
holiday.jpg  ->  Images/holiday.jpg
logo.png  ->  Images/logo.png
minutes.docx  ->  Documents/minutes.docx
notes.txt  ->  Documents/notes.txt
photo2.jpeg  ->  Images/photo2.jpeg
readme  ->  Other/readme
sales.csv  ->  Spreadsheets/sales.csv
setup.exe  ->  Other/setup.exe
song.mp3  ->  Audio/song.mp3
Dry run, would move 12 files (Archives: 1, Audio: 1, Documents:
3, Images: 3, Other: 2, Spreadsheets: 2).
```

Real run (last line only; the file list is identical):

```
Moved 12 files (Archives: 1, Audio: 1, Documents: 3, Images: 3,
Other: 2, Spreadsheets: 2).
```

**How to run it** (from the `source` folder):

```
py project02_organise_by_type.py sample_data/downloads
py project02_organise_by_type.py sample_data/downloads --apply
```

These commands change or create files. To repeat a test, delete the `sample_data` folder and run `py make_sample_data.py` again.

**Common errors and fixes:**

- **Error: ... is not a folder.** Check the path. On Windows, wrap paths with spaces in double quotes.
- **A file lands in Other.** Add its extension to CATEGORIES.

**Safe-use notes:** Never point it at a system folder or a project folder with code. Files are moved, not copied, so take a backup first (Project 4) for anything you cannot replace.

**Optional challenge:** Add a date-based mode that puts files into YYYY-MM sub-folders inside each category.

## Project 3. Duplicate file detection

**Problem:** Copies of the same file are scattered over a folder tree and waste space. You want a list, not an automatic clean-up.

**Expected result:** A text report listing groups of files with identical content and the bytes that could be freed. Nothing is deleted or moved.

**Required tools:** Python standard library only.

**Installation:** nothing to install. Use `py` on Windows or `python3` on macOS and Linux to run it.

**Complete source code** (`project03_find_duplicates.py`):

```
"""Project 3: find duplicate files by content (SHA-256). Report
only:
this script never deletes, moves or renames anything."""
import argparse
import hashlib
from collections import defaultdict
from pathlib import Path


def file_hash(path, chunk_size=1024 * 1024):
    digest = hashlib.sha256()
    with path.open("rb") as fh:
        # Read in chunks so a huge file does not fill the memory.
        while chunk := fh.read(chunk_size):
            digest.update(chunk)
    return digest.hexdigest()


def find_duplicates(folder):
    by_size = defaultdict(list)
    for path in folder.rglob("*"):
        if path.is_file():
            by_size[path.stat().st_size].append(path)
    groups = defaultdict(list)
    for size, paths in by_size.items():
        if len(paths) < 2:  # a file with a unique size cannot be
```

```
a duplicate
            continue
        for path in paths:
            try:
                groups[file_hash(path)].append(path)
            except OSError as error:
                print(f"Skipped {path}: {error}")
    return [sorted(g) for g in groups.values() if len(g) > 1]


def main(argv=None):
    parser = argparse.ArgumentParser(description="Report
duplicate files.")
    parser.add_argument("folder", type=Path)
    parser.add_argument("--report", type=Path,
default=Path("duplicates_report.txt"))
    args = parser.parse_args(argv)
    if not args.folder.is_dir():
        print(f"Error: {args.folder} is not a folder.")
        return 1

    groups = find_duplicates(args.folder)
    lines, wasted = [], 0
    for number, group in enumerate(groups, start=1):
        size = group[0].stat().st_size
        wasted += size * (len(group) - 1)
        lines.append(f"Group {number} ({size} bytes each):")
        lines.extend(f"  {p.relative_to(args.folder)}" for p in
group)
    lines.append(f"{len(groups)} duplicate groups, {wasted} bytes
could be "
                 "freed. Nothing was deleted.")
    args.report.write_text("\n".join(lines) + "\n",
encoding="utf-8")
    print("\n".join(lines))
    print(f"Report saved to {args.report}")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

**Code explanation:**

- Files are first grouped by size. Two files with different sizes cannot be identical, so most files are never read.
- `file_hash` reads 1 MB at a time and feeds a SHA-256 digest. Identical content gives an identical digest.
- The `:=` walrus operator reads a chunk and tests it in one line; the loop ends when `read` returns empty bytes.
- The script only writes the report file. There is no delete, move or rename call anywhere in it.

**Sample input:** Folder `sample_data/dupes`: two identical invoice text files in different sub-folders, two identical 2 KB binary files, and two files that look alike but differ.

**Sample output** (real output from my test run, Windows 11, Python 3.14.4, 2026-11-07):

```
Group 1 (2048 bytes each):
  c\photo.bin
  photo_again.bin
Group 2 (18 bytes each):
  a\invoice_march.txt
  b\invoice_march_copy.txt
2 duplicate groups, 2066 bytes could be freed. Nothing was
deleted.
Report saved to duplicates_report.txt
```

**How to run it** (from the `source` folder):

```
py project03_find_duplicates.py sample_data/dupes
```

**Common errors and fixes:**

- **PermissionError while hashing.** The script prints Skipped and goes on. Run it on a folder you own.
- **It is slow.** Large folders take time because files of equal size are read in full. Try a smaller folder first.

**Safe-use notes:** A match means identical bytes, but only you can decide which copy to keep. Delete by hand, after looking at the report, or move to the recycle bin yourself.

**Optional challenge:** Add a --min-size option to ignore files smaller than a given number of bytes.

## Project 4. Automated file backup with timestamps

**Problem:** You want a dated copy of a project folder before you change it, without ever overwriting an earlier backup.

**Expected result:** A new folder named projectname_YYYYMMDD_HHMMSS containing a full copy. The source is untouched.

**Required tools:** Python standard library only.

**Installation:** nothing to install. Use `py` on Windows or `python3` on macOS and Linux to run it.

**Complete source code** (`project04_backup_folder.py`):

```
"""Project 4: copy a folder into a new time-stamped backup
folder.
It never overwrites an existing backup and never touches the
source."""
import argparse
import shutil
from datetime import datetime
from pathlib import Path


def main(argv=None):
    parser = argparse.ArgumentParser(description="Timestamped
folder backup.")
    parser.add_argument("source", type=Path)
    parser.add_argument("destination", type=Path,
                        help="folder that will hold the backups")
    parser.add_argument("--dry-run", action="store_true")
    args = parser.parse_args(argv)

    source = args.source.resolve()
    destination = args.destination.resolve()
    if not source.is_dir():
        print(f"Error: {source} is not a folder.")
        return 1
    # A backup inside the source would copy itself again on the
next run.
```

```
    if destination == source or source in destination.parents:
        print("Error: the destination must be outside the source
folder.")
        return 1

    stamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    target = destination / f"{source.name}_{stamp}"
    files = [p for p in source.rglob("*") if p.is_file()]
    total = sum(p.stat().st_size for p in files)
    print(f"Source: {source.name} ({len(files)} files, {total}
bytes)")
    print(f"Backup would be: {target.name}")
    if args.dry_run:
        print("Dry run: nothing copied.")
        return 0

    destination.mkdir(parents=True, exist_ok=True)
    try:
        # copytree refuses to run if target exists, so no
overwrite is possible.
        shutil.copytree(source, target)
    except (OSError, shutil.Error) as error:
        print(f"Backup failed: {error}")
        return 1
    copied = [p for p in target.rglob("*") if p.is_file()]
    if len(copied) != len(files):
        print("Warning: file counts differ. Check the backup by
hand.")
        return 1
    print(f"Backup complete: {len(copied)} files copied to
{target.name}")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

**Code explanation:**

- The destination is resolved to an absolute path and refused if it sits inside the source. Otherwise each backup would contain the previous one.
- `strftime("%Y%m%d_%H%M%S")` makes names that sort in time order.

- `shutil.copytree` raises an error if the target already exists, which gives you no-overwrite behaviour for free.
- After copying, the script compares file counts and warns if they differ. This is a basic check, not proof that every byte is identical.

**Sample input:** Folder `sample_data/project_files` (four files, 155,050 bytes).

**Sample output** (real output from my test run, Windows 11, Python 3.14.4, 2026-11-07):

Dry run:

```
Source: project_files (4 files, 155050 bytes)
Backup would be: project_files_20261107_012621
Dry run: nothing copied.
```

Real backup:

```
Source: project_files (4 files, 155050 bytes)
Backup would be: project_files_20261107_012624
Backup complete: 4 files copied to project_files_20261107_012624
```

Destination inside the source (refused):

```
Error: the destination must be outside the source folder.
```

**How to run it** (from the `source` folder):

```
py project04_backup_folder.py sample_data/project_files backups -
-dry-run
py project04_backup_folder.py sample_data/project_files backups
py project04_backup_folder.py sample_data/project_files
sample_data/project_files/inside
```

**Common errors and fixes:**

- **FileExistsError / Backup failed.** Two runs in the same second. Wait a second and run again.
- **Backup failed: [Errno 28] No space left.** The target drive is full. Choose another drive.

**Safe-use notes:** A backup on the same disk does not protect you from disk failure. Keep a second copy elsewhere. Open a backup now and then to confirm it is usable.

**Optional challenge:** Add --keep N so the script lists backups older than the newest N (but only lists them; deleting stays manual).

## Project 5. Folder size report

**Problem:** The disk is nearly full and you do not know which folder is responsible.

**Expected result:** A total, a size per top-level sub-folder, and the largest files. Read only.

**Required tools:** Python standard library only.

**Installation:** nothing to install. Use `py` on Windows or `python3` on macOS and Linux to run it.

**Complete source code** (`project05_folder_size_report.py`):

```
"""Project 5: show how much space each sub-folder uses and the
biggest files.
Read only: nothing is changed."""
import argparse
from pathlib import Path


def human(size):
    for unit in ("B", "KB", "MB", "GB"):
        if size < 1024 or unit == "GB":
            return f"{size:.0f} {unit}" if unit == "B" else f"
{size:.1f} {unit}"
        size /= 1024


def main(argv=None):
    parser = argparse.ArgumentParser(description="Folder size
report.")
    parser.add_argument("folder", type=Path)
    parser.add_argument("--top", type=int, default=3,
help="biggest files")
    args = parser.parse_args(argv)
    if not args.folder.is_dir():
        print(f"Error: {args.folder} is not a folder.")
        return 1

    per_folder, files = {}, []
```

```
    for path in args.folder.rglob("*"):
        if not path.is_file():
            continue
        try:
            size = path.stat().st_size
        except OSError:  # e.g. a file that vanished while we
were scanning
            continue
        parts = path.relative_to(args.folder).parts
        top = parts[0] if len(parts) > 1 else "(files in the top
folder)"
        per_folder[top] = per_folder.get(top, 0) + size
        files.append((size, path))

    print(f"Total: {human(sum(per_folder.values()))} in
{len(files)} files")
    print("By sub-folder, largest first:")
    for name, size in sorted(per_folder.items(), key=lambda x: -
x[1]):
        print(f"  {human(size):>9}  {name}")
    print(f"Top {args.top} files:")
    for size, path in sorted(files, reverse=True)[: args.top]:
        print(f"  {human(size):>9}
{path.relative_to(args.folder)}")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

**Code explanation:**

- `human` turns a byte count into KB, MB or GB by dividing by 1024 until the number is small.
- `path.relative_to(args.folder).parts[0]` gives the first folder name below the start folder, used as the group key.
- `try/except OSError` around `stat()` covers files that disappear while the scan runs.
- `sorted(files, reverse=True)` sorts by size first because each item is a (size, path) pair.

**Sample input:** Folder `sample_data/project_files`.

**Sample output** (real output from my test run, Windows 11, Python 3.14.4, 2026-11-07):

```
Total: 151.4 KB in 4 files
By sub-folder, largest first:
   149.6 KB  data
     1.6 KB  docs
      250 B  (files in the top folder)
Top 3 files:
   146.5 KB  data\big.bin
     3.1 KB  data\numbers.csv
     1.6 KB  docs\guide.md
```

**How to run it** (from the `source` folder):

```
py project05_folder_size_report.py sample_data/project_files
```

**Common errors and fixes:**

- **Output shows 0 files.** The folder is empty or the path is wrong.
- **Numbers differ from Windows Explorer.** Explorer may show size on disk, which rounds up to disk blocks.

**Safe-use notes:** The script only reads. On very large drives a full scan can take minutes.

**Optional challenge:** Write the sub-folder table to a CSV so you can compare sizes month to month.

## Project 6. Image file organisation by date

**Problem:** Hundreds of photos sit in one folder. You want YEAR/YEAR-MONTH folders.

**Expected result:** Image files copied (or moved with --move) into folders built from each file's modified date.

**Required tools:** Python standard library only.

**Installation:** nothing to install. Use `py` on Windows or `python3` on macOS and Linux to run it.

**Complete source code** (`project06_photo_organiser.py`):

```
"""Project 6: copy image files into YEAR/YEAR-MONTH folders using
the file's
modified time. Copies by default, so the originals stay where
they are."""
import argparse
import shutil
from datetime import datetime
from pathlib import Path

IMAGE_TYPES = {".jpg", ".jpeg", ".png", ".gif", ".heic"}


def main(argv=None):
    parser = argparse.ArgumentParser(description="Organise photos
by date.")
    parser.add_argument("source", type=Path)
    parser.add_argument("destination", type=Path)
    parser.add_argument("--move", action="store_true",
                        help="move instead of copy")
    parser.add_argument("--dry-run", action="store_true")
    args = parser.parse_args(argv)
    if not args.source.is_dir():
        print(f"Error: {args.source} is not a folder.")
        return 1

    done = skipped = 0
    for path in sorted(args.source.iterdir()):
```

```
        if not path.is_file() or path.suffix.lower() not in
IMAGE_TYPES:
            continue
        # Modified time is a stand-in for "date taken"; see the
book text.
        taken = datetime.fromtimestamp(path.stat().st_mtime)
        folder = args.destination / str(taken.year) /
taken.strftime("%Y-%m")
        target = folder / path.name
        if target.exists():
            print(f"Skipped {path.name}: already in
{folder.name}")
            skipped += 1
            continue
        print(f"{path.name} ->
{target.relative_to(args.destination)}")
        if not args.dry_run:
            folder.mkdir(parents=True, exist_ok=True)
            # copy2 keeps the modified time, so a re-run files it
the same way.
            (shutil.move if args.move else shutil.copy2)(path,
target)
        done += 1
    verb = "Would handle" if args.dry_run else ("Moved" if
args.move else "Copied")
    print(f"{verb} {done} images, skipped {skipped}.")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

**Code explanation:**

- The date comes from `st_mtime`, the modified time. For a camera file this is often the capture time, but copying or editing can change it. Reading the true capture date needs EXIF data and a library such as Pillow, which this project does not use.
- `strftime("%Y-%m")` builds the month folder name.
- Copying is the default and `copy2` keeps the original modified time, so a second run files each picture in the same place.

- A file that already exists at the target is skipped, which makes the script safe to re-run.

**Sample input:** Folder `sample_data/photos`: five dummy image files with modified dates from August to October 2026, and one text file that must be ignored. The files are not real pictures; only names and dates matter here.

**Sample output** (real output from my test run, Windows 11, Python 3.14.4, 2026-11-07):

Dry run:

```
a.jpg -> 2026\2026-08\a.jpg
b.jpg -> 2026\2026-08\b.jpg
c.png -> 2026\2026-09\c.png
d.jpeg -> 2026\2026-10\d.jpeg
e.jpg -> 2026\2026-10\e.jpg
Would handle 5 images, skipped 0.
```

Real run (last line only):

```
Copied 5 images, skipped 0.
```

Second real run:

```
Skipped a.jpg: already in 2026-08
Skipped b.jpg: already in 2026-08
Skipped c.png: already in 2026-09
Skipped d.jpeg: already in 2026-10
Skipped e.jpg: already in 2026-10
Copied 0 images, skipped 5.
```

**How to run it** (from the `source` folder):

```
py project06_photo_organiser.py sample_data/photos photos_sorted
--dry-run
```

```
py project06_photo_organiser.py sample_data/photos photos_sorted
py project06_photo_organiser.py sample_data/photos photos_sorted
```

These commands change or create files. To repeat a test, delete the `sample_data` folder and run `py make_sample_data.py` again.

**Common errors and fixes:**

- **Photos land in the wrong month.** Modified time was changed by an edit or copy. Use EXIF dates in the challenge.
- **HEIC files are skipped.** Add the extension .heic if it is not listed; it is in the sample set.

**Safe-use notes:** Copy mode leaves the originals alone. Use --move only after you have checked the copies.

**Optional challenge:** Install Pillow and read the EXIF capture date, falling back to modified time when EXIF is missing.

CHAPTER 5

# CSV processing

A CSV (comma separated values) file is plain text with one record per line and fields separated by commas. Spreadsheets, shops, banks and databases all export it. Python's `csv` module handles quotes and commas inside fields correctly, which is why you should not split lines on commas yourself.

Three habits for the following projects. Open files with `newline=""` and an explicit encoding. Use `utf-8-sig` when reading, which also copes with the invisible marker Excel adds to some UTF-8 files. Read with `DictReader` so columns are found by name, not position.

```
import csv
with open("expenses.csv", newline="", encoding="utf-8-sig") as
fh:
    for row in csv.DictReader(fh):
        print(row["category"], row["amount"])
```

## Project 7. CSV cleanup

**Problem:** A CSV export has stray spaces, mixed capitalisation, money written as $1,200.50, dates in four formats, blank lines and duplicate rows.

**Expected result:** clean.csv with tidy rows, and rejects.csv listing every row that could not be fixed, with the reason.

**Required tools:** Python standard library only.

**Installation:** nothing to install. Use `py` on Windows or `python3` on macOS and Linux to run it.

**Complete source code** (`project07_csv_cleanup.py`):

```
"""Project 7: clean a messy CSV. Writes a clean file and a
rejects file;
the original is never changed."""
import argparse
import csv
from datetime import datetime
from pathlib import Path

DATE_FORMATS = ["%Y-%m-%d", "%d/%m/%Y", "%d-%m-%Y", "%b %d %Y"]


def parse_date(text):
    for fmt in DATE_FORMATS:
        try:
            return datetime.strptime(text, fmt).strftime("%Y-%m-
%d")
        except ValueError:
            pass
    raise ValueError(f"unknown date '{text}'")


def clean_row(row):
    # Header names and values may carry stray spaces: normalise
both.
    row = {k.strip().lower(): (v or "").strip() for k, v in
row.items()}
```

```
    name = " ".join(row["name"].split()).title()
    region = row["region"].title()
    amount = row["amount"].replace("$", "").replace(",", "")
    return {"name": name, "region": region,
            "amount": f"{float(amount):.2f}", "date":
parse_date(row["date"])}


def main(argv=None):
    parser = argparse.ArgumentParser(description="Clean a CSV
file.")
    parser.add_argument("input", type=Path)
    parser.add_argument("--out", type=Path,
default=Path("clean.csv"))
    parser.add_argument("--rejects", type=Path,
default=Path("rejects.csv"))
    args = parser.parse_args(argv)
    if not args.input.is_file():
        print(f"Error: {args.input} not found.")
        return 1

    clean, rejects, seen = [], [], set()
    with args.input.open(newline="", encoding="utf-8-sig") as fh:
        for line_no, row in enumerate(csv.DictReader(fh),
start=2):
            if not any((v or "").strip() for v in row.values()):
                continue  # blank line
            try:
                item = clean_row(row)
            except (ValueError, KeyError) as error:
                rejects.append({"line": line_no, "reason": error,
                                "raw": "|".join(str(v) for v in
row.values())})
                continue
            key = tuple(item.values())
            if key in seen:
                rejects.append({"line": line_no, "reason":
"duplicate row",
                                "raw": "|".join(key)})
                continue
            seen.add(key)
            clean.append(item)

    for path, rows, fields in [(args.out, clean, ["name",
"region", "amount", "date"]),
                               (args.rejects, rejects, ["line",
```

```
"reason", "raw"])]:
        with path.open("w", newline="", encoding="utf-8") as fh:
            writer = csv.DictWriter(fh, fieldnames=fields)
            writer.writeheader()
            writer.writerows(rows)
    print(f"Clean rows: {len(clean)}  Rejected: {len(rejects)}")
    for r in rejects:
        print(f"  line {r['line']}: {r['reason']}")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

**Code explanation:**

- `DictReader` gives each row as a dictionary keyed by the header. `.strip().lower()` on the keys copes with a header such as `Region`.
- `" ".join(text.split())` collapses any run of spaces into one, then `.title()` fixes the capitalisation.
- `parse_date` tries each format in `DATE_FORMATS`. Order matters: 03/10/2026 is read as day/month, so 3 October. If your data uses month/day, change the list.
- Rows that raise `ValueError` or `KeyError` go to the rejects list instead of stopping the run.
- `seen` is a set of finished rows, which detects exact duplicates after cleaning.

**Sample input:** `sample_data/sales_messy.csv`, 8 data lines (one blank, one duplicate after cleaning, one amount `abc`, one text date).

**Sample output** (real output from my test run, Windows 11, Python 3.14.4, 2026-11-07):

```
Clean rows: 4  Rejected: 3
  line 5: duplicate row
  line 7: could not convert string to float: 'abc'
  line 9: unknown date 'not a date'
```

**How to run it** (from the `source` folder):

```
py project07_csv_cleanup.py sample_data/sales_messy.csv
```

**Common errors and fixes:**

- **KeyError: 'name'.** The header is missing or named differently. Edit the column names in clean_row.
- **UnicodeDecodeError.** The file is not UTF-8. Try encoding="cp1252" in the open call.

**Safe-use notes:** The input file is only read. Look at rejects.csv before you trust the clean file; silently dropping rows is how reports go wrong.

**Optional challenge:** Add a --date-order option so the user can choose day-first or month-first.

## Project 8. Splitting a large CSV

**Problem:** A CSV is too big for a tool that accepts a limited number of rows, or you want to hand parts to different people.

**Expected result:** Files named name_part001.csv, name_part002.csv and so on, each with the header row repeated.

**Required tools:** Python standard library only.

**Installation:** nothing to install. Use `py` on Windows or `python3` on macOS and Linux to run it.

**Complete source code** (`project08_split_csv.py`):

```
"""Project 8: split a big CSV into smaller files, repeating the
header."""
import argparse
import csv
from pathlib import Path


def main(argv=None):
    parser = argparse.ArgumentParser(description="Split a CSV
file.")
    parser.add_argument("input", type=Path)
    parser.add_argument("outdir", type=Path)
    parser.add_argument("--rows", type=int, default=10,
help="rows per part")
    args = parser.parse_args(argv)
    if not args.input.is_file():
        print(f"Error: {args.input} not found.")
        return 1
    if args.rows < 1:
        print("Error: --rows must be 1 or more.")
        return 1
    if args.outdir.exists() and any(args.outdir.iterdir()):
        print(f"Error: {args.outdir} is not empty. Choose a new
folder.")
        return 1
    args.outdir.mkdir(parents=True, exist_ok=True)
```

```
    parts = total = 0
    with args.input.open(newline="", encoding="utf-8-sig") as fh:
        reader = csv.reader(fh)
        header = next(reader, None)
        if header is None:
            print("Error: the file is empty.")
            return 1
        batch = []
        for row in reader:
            batch.append(row)
            if len(batch) == args.rows:
                parts += 1
                write_part(args.outdir, args.input.stem, parts,
header, batch)
                total += len(batch)
                batch = []
        if batch:  # the last, shorter part
            parts += 1
            write_part(args.outdir, args.input.stem, parts,
header, batch)
            total += len(batch)
    print(f"Split {total} data rows into {parts} files in
{args.outdir}")
    return 0


def write_part(outdir, stem, number, header, rows):
    path = outdir / f"{stem}_part{number:03d}.csv"
    with path.open("w", newline="", encoding="utf-8") as fh:
        writer = csv.writer(fh)
        writer.writerow(header)
        writer.writerows(rows)
    print(f"  {path.name}: {len(rows)} rows")


if __name__ == "__main__":
    raise SystemExit(main())
```

**Code explanation:**

- The file is read row by row, so memory use stays small even for large files.
- `batch` collects rows; when it reaches `--rows`, `write_part` saves it and the list is emptied.

- After the loop, a shorter final batch is written too. Forgetting this is the classic off-by-one bug.
- The script refuses an output folder that already contains files.

**Sample input:** `sample_data/orders.csv` with 25 order rows.

**Sample output** (real output from my test run, Windows 11, Python 3.14.4, 2026-11-07):

```
  orders_part001.csv: 10 rows
  orders_part002.csv: 10 rows
  orders_part003.csv: 5 rows
Split 25 data rows into 3 files in orders_parts
```

**How to run it** (from the `source` folder):

```
py project08_split_csv.py sample_data/orders.csv orders_parts --
rows 10
```

**Common errors and fixes:**

- **Error: ... is not empty.** Choose a new output folder name.
- **Parts have odd line breaks.** Open files with newline="" as the script does; do not remove it.

**Safe-use notes:** The original is not changed. Check the row total printed at the end against the original.

**Optional challenge:** Split by the value of a column instead, for example one file per customer.

## Project 9. Simple expense tracking

**Problem:** You keep a CSV of expenses (date, category, description, amount) and want monthly and category totals.

**Expected result:** Printed totals by month and by category with percentages, and expense_summary.csv.

**Required tools:** Python standard library only.

**Installation:** nothing to install. Use `py` on Windows or `python3` on macOS and Linux to run it.

**Complete source code** (`project09_expense_tracker.py`):

```
"""Project 9: read expenses from a CSV and print totals by
category and month.
Money uses Decimal, not float, so cents never drift."""
import argparse
import csv
from collections import defaultdict
from decimal import Decimal, InvalidOperation
from pathlib import Path


def main(argv=None):
    parser = argparse.ArgumentParser(description="Expense
summary.")
    parser.add_argument("input", type=Path)
    parser.add_argument("--out", type=Path,
default=Path("expense_summary.csv"))
    args = parser.parse_args(argv)
    if not args.input.is_file():
        print(f"Error: {args.input} not found.")
        return 1

    by_category = defaultdict(Decimal)
    by_month = defaultdict(Decimal)
    bad = 0
    with args.input.open(newline="", encoding="utf-8-sig") as fh:
        for line_no, row in enumerate(csv.DictReader(fh),
start=2):
```

```
            try:
                amount = Decimal(row["amount"])
                month = row["date"][:7]  # "2026-10-05" -> "2026-
10"
                by_category[row["category"].strip().title()] +=
amount
                by_month[month] += amount
            except (InvalidOperation, KeyError, TypeError):
                bad += 1
                print(f"Skipped line {line_no}: unreadable row")

    grand = sum(by_month.values(), Decimal("0"))
    print("By month:")
    for month, total in sorted(by_month.items()):
        print(f"  {month}  {total:>10.2f}")
    print("By category (largest first):")
    for cat, total in sorted(by_category.items(), key=lambda x: -
x[1]):
        share = total / grand * 100 if grand else Decimal(0)
        print(f"  {cat:<10} {total:>10.2f}  {share:>5.1f}%")
    print(f"Grand total: {grand:.2f}  (skipped rows: {bad})")

    with args.out.open("w", newline="", encoding="utf-8") as fh:
        writer = csv.writer(fh)
        writer.writerow(["type", "name", "total"])
        writer.writerows(("month", m, f"{t:.2f}") for m, t in
sorted(by_month.items()))
        writer.writerows(("category", c, f"{t:.2f}") for c, t in
sorted(by_category.items()))
    print(f"Summary saved to {args.out}")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

**Code explanation:**

- `Decimal` is used for money. With plain floats, sums such as 0.1 + 0.2 produce tiny errors; Decimal keeps exact cents.
- `defaultdict(Decimal)` starts every new key at zero, so `+=` works on first use.

- `row["date"][:7]` slices "2026-10-05" to "2026-10". It relies on ISO dates; clean other formats with Project 7 first.
- The `{total:>10.2f}` format right-aligns numbers in a column with two decimals.

**Sample input:** `sample_data/expenses.csv`: 18 invented expenses for September and October 2026.

**Sample output** (real output from my test run, Windows 11, Python 3.14.4, 2026-11-07):

```
By month:
  2026-09     1288.00
  2026-10     1274.00
By category (largest first):
  Rent          1816.00   70.9%
  Food           424.00   16.5%
  Health         142.00    5.5%
  Transport      135.00    5.3%
  Software        45.00    1.8%
Grand total: 2562.00  (skipped rows: 0)
Summary saved to expense_summary.csv
```

**How to run it** (from the `source` folder):

```
py project09_expense_tracker.py sample_data/expenses.csv
```

**Common errors and fixes:**

- **Skipped line N: unreadable row.** The amount has a currency symbol or comma. Clean it with Project 7.
- **Totals look too small.** Rows with a missing amount are skipped and counted at the end.

**Safe-use notes:** Read only apart from the summary file. These are sample numbers, not advice about any budget.

**Optional challenge:** Add a monthly budget per category in a second CSV and flag categories that exceed it.

## Project 10. Contact list de-duplication

**Problem:** A contact export holds the same person twice, with different spellings but the same email or phone.

**Expected result:** contacts_unique.csv, a contacts_dropped.csv showing which earlier contact each dropped row matched, and a printed list.

**Required tools:** Python standard library only.

**Installation:** nothing to install. Use `py` on Windows or `python3` on macOS and Linux to run it.

**Complete source code** (`project10_contact_dedupe.py`):

```
"""Project 10: find duplicate contacts by email or phone number.
Writes a de-duplicated copy and a list of what was dropped; the
original
file is untouched."""
import argparse
import csv
import re
from pathlib import Path


def phone_key(phone):
    digits = re.sub(r"\D", "", phone)
    return digits[-7:] if len(digits) >= 7 else ""  # compare the
last 7 digits


def main(argv=None):
    parser = argparse.ArgumentParser(description="Contact list
de-duplicator.")
    parser.add_argument("input", type=Path)
    parser.add_argument("--out", type=Path,
default=Path("contacts_unique.csv"))
    parser.add_argument("--dropped", type=Path,
default=Path("contacts_dropped.csv"))
    args = parser.parse_args(argv)
    if not args.input.is_file():
```

```
        print(f"Error: {args.input} not found.")
        return 1

    seen_email, seen_phone = {}, {}
    keep, dropped = [], []
    with args.input.open(newline="", encoding="utf-8-sig") as fh:
        reader = csv.DictReader(fh)
        fields = reader.fieldnames
        for row in reader:
            email = row["email"].strip().lower()
            phone = phone_key(row["phone"])
            match = seen_email.get(email) if email else None
            match = match or (seen_phone.get(phone) if phone else
None)
            if match:
                dropped.append({**row, "duplicate_of": match})
                continue
            keep.append(row)
            if email:
                seen_email[email] = row["name"]
            if phone:
                seen_phone[phone] = row["name"]

    with args.out.open("w", newline="", encoding="utf-8") as fh:
        writer = csv.DictWriter(fh, fieldnames=fields)
        writer.writeheader()
        writer.writerows(keep)
    with args.dropped.open("w", newline="", encoding="utf-8") as
fh:
        writer = csv.DictWriter(fh, fieldnames=fields +
["duplicate_of"])
        writer.writeheader()
        writer.writerows(dropped)
    print(f"Kept {len(keep)}, flagged {len(dropped)} as
duplicates:")
    for row in dropped:
        print(f"  {row['name']} looks like
{row['duplicate_of']}")
    print("Review the dropped list before trusting it: matches
can be wrong.")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

**Code explanation:**

- Emails are lowercased before comparing, so ALICE@example.com matches alice@example.com.
- `phone_key` removes everything but digits and compares the last seven, so +1 555 0101 matches 555-0101.
- Two dictionaries remember which name first used each email or phone, so the report can say who a row duplicates.
- Rows without an email or a usable phone are never matched on that field. Otherwise all blanks would collapse into one.

**Sample input:** `sample_data/contacts.csv`, 7 rows. Note Robert Smith has a different email from Bob Smith but the same phone.

**Sample output** (real output from my test run, Windows 11, Python 3.14.4, 2026-11-07):

```
Kept 4, flagged 3 as duplicates:
  A. Johnson looks like Alice Johnson
  Robert Smith looks like Bob Smith
  Dan Lee looks like Dan Lee
Review the dropped list before trusting it: matches can be wrong.
```

**How to run it** (from the `source` folder):

```
py project10_contact_dedupe.py sample_data/contacts.csv
```

**Common errors and fixes:**

- **KeyError: 'email'.** Your file has different column names. Change them in the script.
- **Two different people were merged.** Shared phones (an office line) cause this. Always read the dropped file.

**Safe-use notes:** The original file is untouched and nothing is merged automatically beyond dropping the later row. Matching is a guess; a person must confirm it.

**Optional challenge:** Keep the row with the most filled-in fields instead of always keeping the first.

## Project 11. Data validation report

**Problem:** Before importing a file into another system you want to know which rows break the rules.

**Expected result:** A CSV report of every problem (line, field, value, reason) and an exit code of 2 when problems exist.

**Required tools:** Python standard library only.

**Installation:** nothing to install. Use `py` on Windows or `python3` on macOS and Linux to run it.

**Complete source code** (`project11_validation_report.py`):

```
"""Project 11: check each row of a CSV against simple rules and
write a
report of every problem. The data file is only read, never
changed."""
import argparse
import csv
import re
from datetime import datetime
from pathlib import Path

EMAIL = re.compile(r"^[^@\s]+@[^@\s]+\.[^@\s]+$")  # a sanity
check, not RFC
STATUSES = {"paid", "pending", "shipped", "cancelled"}


def check_row(row):
    problems = []
    if not row["order_id"].strip():
        problems.append(("order_id", "missing"))
    if not EMAIL.match(row["email"].strip()):
        problems.append(("email", "does not look like an email
address"))
    try:
        if float(row["amount"]) <= 0:
            problems.append(("amount", "must be greater than
zero"))
    except ValueError:
```

```
        problems.append(("amount", "not a number"))
    try:
        datetime.strptime(row["date"], "%Y-%m-%d")
    except ValueError:
        problems.append(("date", "not a real YYYY-MM-DD date"))
    if row["status"].strip().lower() not in STATUSES:
        problems.append(("status", f"must be one of
{sorted(STATUSES)}"))
    return problems


def main(argv=None):
    parser = argparse.ArgumentParser(description="CSV validation
report.")
    parser.add_argument("input", type=Path)
    parser.add_argument("--report", type=Path,
default=Path("validation_report.csv"))
    args = parser.parse_args(argv)
    if not args.input.is_file():
        print(f"Error: {args.input} not found.")
        return 1

    findings, rows = [], 0
    with args.input.open(newline="", encoding="utf-8-sig") as fh:
        reader = csv.DictReader(fh)
        needed = {"order_id", "email", "amount", "date",
"status"}
        if not needed.issubset(reader.fieldnames or []):
            print(f"Error: the file needs these columns:
{sorted(needed)}")
            return 1
        for line_no, row in enumerate(reader, start=2):
            rows += 1
            for field, message in check_row(row):
                findings.append([line_no, field, row[field],
message])

    with args.report.open("w", newline="", encoding="utf-8") as
fh:
        writer = csv.writer(fh)
        writer.writerow(["line", "field", "value", "problem"])
        writer.writerows(findings)
    bad_rows = len({f[0] for f in findings})
    print(f"Checked {rows} rows: {bad_rows} with problems, "
          f"{len(findings)} problems in total.")
    for line_no, field, value, message in findings:
```

```
        print(f"  line {line_no}: {field} '{value}' - {message}")
    return 0 if not findings else 2  # exit code 2 lets a
scheduler notice


if __name__ == "__main__":
    raise SystemExit(main())
```

**Code explanation:**

- `check_row` returns a list of problems rather than stopping at the first, so one run shows everything.
- The email pattern checks the shape `something@something.something`. It cannot tell whether the mailbox exists.
- `datetime.strptime` rejects impossible dates such as month 13.
- Line numbers start at 2 because line 1 is the header; they match what you see in a spreadsheet.

**Sample input:** `sample_data/orders_to_check.csv`, 6 rows with one valid row and five different faults.

**Sample output** (real output from my test run, Windows 11, Python 3.14.4, 2026-11-07):

```
Checked 6 rows: 5 with problems, 5 problems in total.
  line 3: email 'not-an-email' - does not look like an email
address
  line 4: amount '-5' - must be greater than zero
  line 5: date '2026-13-40' - not a real YYYY-MM-DD date
  line 6: status 'shipped??' - must be one of ['cancelled',
'paid', 'pending', 'shipped']
  line 7: order_id '' - missing
```

**How to run it** (from the `source` folder):

```
py project11_validation_report.py sample_data/orders_to_check.csv
```

**Common errors and fixes:**

- **Error: the file needs these columns.** Rename the columns or edit the `needed` set.
- **Everything is flagged as a bad date.** Your dates use another format. Change the pattern in strptime.

**Safe-use notes:** Read only. Exit code 2 is useful in schedulers and batch files because they can react to it.

**Optional challenge:** Add a rule that order_id values must be unique.

## Project 12. Invoice number generator

**Problem:** Invoice numbers must be unique and in sequence, even across several runs.

**Expected result:** Numbers such as INV-2026-0001, remembered between runs in a small JSON file and logged in a CSV.

**Required tools:** Python standard library only.

**Installation:** nothing to install. Use `py` on Windows or `python3` on macOS and Linux to run it.

**Complete source code** (`project12_invoice_numbers.py`):

```
"""Project 12: hand out invoice numbers such as INV-2026-0001
that never
repeat, even if you run the script many times. A small JSON file
remembers
the last number used per year; every number is also logged to a
CSV."""
import argparse
import csv
import json
import os
from datetime import date, datetime
from pathlib import Path


def load_state(path):
    if not path.exists():
        return {}
    try:
        return json.loads(path.read_text(encoding="utf-8"))
    except json.JSONDecodeError:
        # Do not guess: a damaged counter could lead to repeated
numbers.
        raise SystemExit(f"{path} is damaged. Fix or restore it
first.")


def save_state(path, state):
```

```
    # Write to a temporary file, then swap: a crash cannot leave
half a file.
    temp = path.with_suffix(".tmp")
    temp.write_text(json.dumps(state, indent=2), encoding="utf-
8")
    os.replace(temp, path)


def main(argv=None):
    parser = argparse.ArgumentParser(description="Invoice number
generator.")
    parser.add_argument("--count", type=int, default=1)
    parser.add_argument("--prefix", default="INV")
    parser.add_argument("--state", type=Path,
default=Path("invoice_state.json"))
    parser.add_argument("--log", type=Path,
default=Path("invoice_log.csv"))
    args = parser.parse_args(argv)
    if not 1 <= args.count <= 500:
        print("Error: --count must be between 1 and 500.")
        return 1

    year = str(date.today().year)
    state = load_state(args.state)
    last = state.get(year, 0)
    new_numbers = [f"{args.prefix}-{year}-{n:04d}"
                   for n in range(last + 1, last + 1 +
args.count)]
    # Save the counter BEFORE using the numbers: losing a number
is harmless,
    # reusing one is not.
    state[year] = last + args.count
    save_state(args.state, state)

    is_new = not args.log.exists()
    with args.log.open("a", newline="", encoding="utf-8") as fh:
        writer = csv.writer(fh)
        if is_new:
            writer.writerow(["invoice_number", "issued_at"])
        for number in new_numbers:
            writer.writerow([number,
datetime.now().isoformat(timespec="seconds")])
    print("Issued:", ", ".join(new_numbers))
    return 0
```

```
if __name__ == "__main__":
    raise SystemExit(main())
```

**Code explanation:**

- The state file stores the last number used per year, so the sequence restarts each January.
- The counter is saved before the numbers are shown or logged. If the script crashes, a number is lost but never reused.
- `os.replace` swaps the temporary file in one step, so a crash cannot leave a half written counter.
- A damaged state file stops the script instead of restarting from 1, because guessing would risk duplicates.

**Sample input:** No input file. The first run creates `invoice_state.json` and `invoice_log.csv`.

**Sample output** (real output from my test run, Windows 11, Python 3.14.4, 2026-11-07):

First run, three numbers:

```
Issued: INV-2026-0001, INV-2026-0002, INV-2026-0003
```

Second run, one number:

```
Issued: INV-2026-0004
```

**How to run it** (from the `source` folder):

```
py project12_invoice_numbers.py --count 3
py project12_invoice_numbers.py
```

**Common errors and fixes:**

- **... is damaged. Fix or restore it first.** Restore the file from a backup, or set the last number by hand.

- **Numbers clash with another computer.** This script is for one machine. Two people need a shared system.

**Safe-use notes:** Do not run two copies at the same moment; there is no file locking. Back up the state file with your other data.

**Optional challenge:** Add a --peek option that shows the next number without using it.

CHAPTER 6

# Excel automation

The `openpyxl` package reads and writes `.xlsx` files without Excel being installed. Know its limits before you rely on it.

- **It does not calculate formulas.** It can write `=SUM(A1:A10)` as text for Excel to evaluate. When a program reads a never-calculated file with `data_only=True`, formula cells come back as `None`. Project 13 demonstrates this with real output. If you need a calculated number in Python, calculate it in Python.
- **It handles `.xlsx` only,** not the old `.xls`.
- **Editing an existing workbook may lose features** such as charts, images or macros that openpyxl does not keep (a commonly reported limit that I did not test). Create new files, or work on a copy.
- **Large files** should be read with `read_only=True`.

The openpyxl documentation I tried (the "simple usage" page) returned an error on 2026-11-07, so the formula point comes from my own test in Project 13, not from a quoted page. Check the current documentation at openpyxl.readthedocs.io.

## Project 13. Excel report generation

**Problem:** You have a sales CSV and want a formatted workbook with a data sheet and a summary sheet.

**Expected result:** An .xlsx file with a styled header, frozen top row, number formats, a totals row with formulas, and a Summary sheet.

**Required tools:** Python standard library plus the `openpyxl` package.

**Installation:** `py -m pip install openpyxl` on Windows (the `py` launcher), `python3 -m pip install openpyxl` on macOS and Linux. Run it inside your virtual environment.

**Complete source code** (`project13_excel_report.py`):

```
"""Project 13: turn a sales CSV into a formatted Excel workbook
with a data
sheet and a summary sheet. Needs: python -m pip install
openpyxl"""
import argparse
import csv
from collections import defaultdict
from pathlib import Path

from openpyxl import Workbook, load_workbook
from openpyxl.styles import Alignment, Font, PatternFill


def main(argv=None):
    parser = argparse.ArgumentParser(description="CSV to Excel
report.")
    parser.add_argument("input", type=Path)
    parser.add_argument("output", type=Path)
    args = parser.parse_args(argv)
    if not args.input.is_file():
        print(f"Error: {args.input} not found.")
        return 1
    if args.output.exists():
        print(f"Error: {args.output} already exists. Choose a new
name.")
        return 1
```

```
    wb = Workbook()
    data = wb.active
    data.title = "Data"
    data.append(["Date", "Region", "Product", "Units", "Unit
price", "Revenue"])
    with args.input.open(newline="", encoding="utf-8-sig") as fh:
        for n, row in enumerate(csv.DictReader(fh), start=2):
            # Real numbers, not text, so Excel can sum and sort
them.
            data.append([row["date"], row["region"],
row["product"],
                         int(row["units"]),
float(row["unit_price"]),
                         f"=D{n}*E{n}"])  # a formula: Excel
calculates it
    last = data.max_row
    data.append(["", "", "", f"=SUM(D2:D{last})", "",
f"=SUM(F2:F{last})"])

    header_fill = PatternFill("solid", start_color="1F4E78")
    for cell in data[1]:
        cell.font = Font(bold=True, color="FFFFFF")
        cell.fill = header_fill
        cell.alignment = Alignment(horizontal="center")
    for cell in data[data.max_row]:
        cell.font = Font(bold=True)
    for row in data.iter_rows(min_row=2, min_col=5, max_col=6):
        for cell in row:
            cell.number_format = "#,##0.00"
    for letter, width in zip("ABCDEF", (12, 10, 12, 8, 12, 12)):
        data.column_dimensions[letter].width = width
    data.freeze_panes = "A2"

    # Python's own totals go on the summary sheet as plain
numbers.
    totals = defaultdict(float)
    with args.input.open(newline="", encoding="utf-8-sig") as fh:
        for row in csv.DictReader(fh):
            totals[row["region"]] += int(row["units"]) *
float(row["unit_price"])
    summary = wb.create_sheet("Summary")
    summary.append(["Region", "Revenue (calculated by Python)"])
    for region, value in sorted(totals.items()):
        summary.append([region, round(value, 2)])
    summary.column_dimensions["A"].width = 14
```

```
    summary.column_dimensions["B"].width = 30
    for cell in summary[1]:
        cell.font = Font(bold=True)
    wb.save(args.output)
    print(f"Saved {args.output} ({last - 1} data rows)")

    # openpyxl writes formulas but does not calculate them. Prove
it:
    check = load_workbook(args.output, data_only=True)["Data"]
    print("Formula cell F2 as openpyxl reads it back:",
check["F2"].value)
    print("Python total revenue:", round(sum(totals.values()),
2))
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

**Code explanation:**

- `Workbook()` makes a new file in memory; nothing exists on disk until `wb.save`.
- Units and prices are converted with `int` and `float` so Excel stores numbers, not text.
- `f"=D{n}*E{n}"` writes a formula as text starting with =. Excel calculates it when the file is opened.
- openpyxl does not calculate formulas. The last lines prove it: reading the saved file with `data_only=True` returns None for F2, because no program has calculated and stored a result yet. The Summary sheet therefore holds values calculated by Python.
- The script refuses to overwrite an existing output file.

**Sample input:** `sample_data/sales/sales_2026_10.csv`, 24 invented sales rows (date, region, product, units, unit_price).

**Sample output** (real output from my test run, Windows 11, Python 3.14.4, 2026-11-07):

```
Saved sales_october.xlsx (24 data rows)
Formula cell F2 as openpyxl reads it back: None
Python total revenue: 2692.95
```

**How to run it** (from the `source` folder):

```
py project13_excel_report.py sample_data/sales/sales_2026_10.csv
sales_october.xlsx
```

**Common errors and fixes:**

- **ModuleNotFoundError: No module named 'openpyxl'.** Install it into the Python you are running: py -m pip install openpyxl.
- **PermissionError on save.** The target workbook is open in Excel. Close it.

**Safe-use notes:** Never write over the only copy of a workbook. This script refuses if the name exists. openpyxl can lose features such as charts or macros when it edits an existing file; this project only creates new files.

**Optional challenge:** Add a chart with openpyxl.chart.BarChart showing revenue per region.

## Project 14. Merging spreadsheets

**Problem:** Each branch sends its own workbook with a Sales sheet. You need one combined sheet.

**Expected result:** merged_branches.xlsx with all rows plus a source_file column. Files with different headers are skipped and reported.

**Required tools:** Python standard library plus the `openpyxl` package.

**Installation:** `py -m pip install openpyxl` on Windows (the `py` launcher), `python3 -m pip install openpyxl` on macOS and Linux. Run it inside your virtual environment.

**Complete source code** (`project14_merge_spreadsheets.py`):

```
"""Project 14: merge the "Sales" sheet of every .xlsx file in a
folder into
one workbook. Files whose headers differ are skipped and
reported.
Needs: python -m pip install openpyxl"""
import argparse
from pathlib import Path

from openpyxl import Workbook, load_workbook


def main(argv=None):
    parser = argparse.ArgumentParser(description="Merge Excel
files.")
    parser.add_argument("folder", type=Path)
    parser.add_argument("output", type=Path)
    parser.add_argument("--sheet", default="Sales")
    args = parser.parse_args(argv)
    if not args.folder.is_dir():
        print(f"Error: {args.folder} is not a folder.")
        return 1
    if args.output.exists():
        print(f"Error: {args.output} already exists. Choose a new
name.")
        return 1
```

```
    merged = Workbook()
    out = merged.active
    out.title = "Merged"
    expected, total, skipped = None, 0, []
    for path in sorted(args.folder.glob("*.xlsx")):
        try:
            # read_only keeps memory low; data_only gives stored
values.
            wb = load_workbook(path, read_only=True,
data_only=True)
        except Exception as error:  # damaged or password
protected file
            skipped.append((path.name, f"cannot open ({error})"))
            continue
        if args.sheet not in wb.sheetnames:
            skipped.append((path.name, f"no sheet called
{args.sheet}"))
            wb.close()
            continue
        rows = list(wb[args.sheet].iter_rows(values_only=True))
        wb.close()
        if not rows:
            skipped.append((path.name, "sheet is empty"))
            continue
        header, body = rows[0], rows[1:]
        if expected is None:
            expected = header
            out.append(list(header) + ["source_file"])
        elif header != expected:
            skipped.append((path.name, f"headers differ:
{list(header)}"))
            continue
        for row in body:
            out.append(list(row) + [path.name])
        total += len(body)
        print(f"Merged {path.name}: {len(body)} rows")

    if expected is None:
        print("Nothing to merge.")
        return 1
    merged.save(args.output)
    print(f"Saved {args.output}: {total} rows")
    for name, reason in skipped:
        print(f"SKIPPED {name}: {reason}")
    return 0
```

```
if __name__ == "__main__":
    raise SystemExit(main())
```

**Code explanation:**

- `load_workbook(..., read_only=True, data_only=True)` opens quickly and returns stored values rather than formulas. If a workbook was never calculated and saved by Excel, formula cells give None.
- The first valid file defines the expected header. Any later file with another header is skipped, not forced in.
- `iter_rows(values_only=True)` yields plain tuples, which are easy to compare and copy.
- The `source_file` column keeps the trail back to the original workbook.

**Sample input:** `sample_data/branches` with `branch_north.xlsx`, `branch_south.xlsx`, `branch_east.xlsx` (header: date, product, units, revenue) and `branch_west_wrong_headers.xlsx` (header: day, item, qty).

**Sample output** (real output from my test run, Windows 11, Python 3.14.4, 2026-11-07):

```
Merged branch_east.xlsx: 4 rows
Merged branch_north.xlsx: 4 rows
Merged branch_south.xlsx: 4 rows
Saved merged_branches.xlsx: 12 rows
SKIPPED branch_west_wrong_headers.xlsx: headers differ: ['day',
'item', 'qty']
```

**How to run it** (from the `source` folder):

```
py project14_merge_spreadsheets.py sample_data/branches
merged_branches.xlsx
```

**Common errors and fixes:**

- **SKIPPED ... no sheet called Sales.** Pass --sheet with the right sheet name.
- **SKIPPED ... cannot open.** The file is damaged, password protected or an old .xls file.

**Safe-use notes:** Source workbooks are opened read only and never changed. Old .xls files are not read by openpyxl; save them as .xlsx first.

**Optional challenge:** Add a check that reports rows whose date falls outside the month you expect.

CHAPTER 7

# PDF and document processing

The `pypdf` package can read text, merge pages and split documents. Its documentation is direct about the limits, and so should this book be.

- A PDF stores where to draw characters, not paragraphs or tables. Extracted text can have odd spacing, wrong order in columns, and broken tables.
- A scanned PDF is a set of images. pypdf cannot read text from images; you need OCR software such as Tesseract. I did not test OCR.
- Quality varies from file to file. Always read a sample of the output before trusting it.
- Password protected files need the password. Do not try to bypass protection on files that are not yours.

Word documents (`.docx`) can be handled with the `python-docx` package. I installed it (version 1.2.0) but did not write or test a project with it, so this book does not teach it.

## Project 15. PDF text extraction

**Problem:** You need the text of a PDF in a plain file for searching or editing.

**Expected result:** A text file with the text of each page under a page marker, and a warning for pages without text.

**Required tools:** Python standard library plus the `pypdf` package.

**Installation:** `py -m pip install pypdf` on Windows (the `py` launcher), `python3 -m pip install pypdf` on macOS and Linux. Run it inside your virtual environment.

**Complete source code** (`project15_pdf_text_extract.py`):

```
"""Project 15: extract the text of a PDF page by page into a .txt
file.
Works only for PDFs that contain real text. Scanned pages need
OCR.
Needs: python -m pip install pypdf"""
import argparse
from pathlib import Path

from pypdf import PdfReader
from pypdf.errors import PyPdfError


def main(argv=None):
    parser = argparse.ArgumentParser(description="Extract text
from a PDF.")
    parser.add_argument("pdf", type=Path)
    parser.add_argument("--out", type=Path, help="default: same
name, .txt")
    args = parser.parse_args(argv)
    if not args.pdf.is_file():
        print(f"Error: {args.pdf} not found.")
        return 1
    out = args.out or args.pdf.with_suffix(".txt")
    if out.exists():
        print(f"Error: {out} already exists. Use --out with a new
name.")
```

```
        return 1

    try:
        reader = PdfReader(args.pdf)
        if reader.is_encrypted:
            print("Error: the PDF is password protected.")
            return 1
        parts, empty, words = [], [], 0
        for number, page in enumerate(reader.pages, start=1):
            text = (page.extract_text() or "").strip()
            if not text:
                empty.append(number)
            words += len(text.split())
            parts.append(f"--- Page {number} ---\n{text}\n")
    except (PyPdfError, OSError) as error:
        print(f"Error: could not read the PDF ({error}).")
        return 1

    out.write_text("\n".join(parts), encoding="utf-8")
    print(f"{len(reader.pages)} pages, about {words} words ->
{out}")
    if empty:
        print(f"No text found on pages {empty}: probably scanned
images. "
              "OCR is needed for those.")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

**Code explanation:**

- `PdfReader` loads the file. `reader.is_encrypted` is checked so a protected file gives a clear message.
- `page.extract_text()` returns the text layer. The pypdf documentation explains that PDFs store positioned characters, not paragraphs, so spacing, columns and tables may come out in a strange order.
- A page with no extracted text is counted in `empty`. Scanned pages are images; pypdf never reads text from images and the documentation points to OCR software such as Tesseract for

that.
- The sample `scanned_like.pdf` is a blank page I built to simulate a page with no text layer. It is not a real scan, and OCR was not tested in this book.

**Sample input:** `sample_data/pdfs/handbook.pdf` (5 pages of generated text) and `sample_data/pdfs/scanned_like.pdf` (one blank page).

**Sample output** (real output from my test run, Windows 11, Python 3.14.4, 2026-11-07):

Text PDF:

```
5 pages, about 45 words -> handbook.txt
```

Page without a text layer:

```
1 pages, about 0 words -> scanned.txt
No text found on pages [1]: probably scanned images. OCR is
needed for those.
```

**How to run it** (from the `source` folder):

```
py project15_pdf_text_extract.py sample_data/pdfs/handbook.pdf --
out handbook.txt
py project15_pdf_text_extract.py
sample_data/pdfs/scanned_like.pdf --out scanned.txt
```

**Common errors and fixes:**

- **Error: ... password protected.** Decrypt it with the password in a tool you trust. This script does not guess.
- **Words glued together or in the wrong order.** A known limit of PDF text extraction. Try a different PDF tool or export from the source program.

**Safe-use notes:** Only process PDFs from sources you trust. Do not try to remove passwords from files you have no right to open.

**Optional challenge:** Search the extracted text for a keyword and print the page numbers where it appears.

## Project 16. Merging PDF files

**Problem:** Several one-page PDFs must become one document in a set order.

**Expected result:** A new PDF containing all pages in the order of the arguments.

**Required tools:** Python standard library plus the `pypdf` package.

**Installation:** `py -m pip install pypdf` on Windows (the `py` launcher), `python3 -m pip install pypdf` on macOS and Linux. Run it inside your virtual environment.

**Complete source code** (`project16_merge_pdfs.py`):

```
"""Project 16: join several PDFs into one, in the order you list
them.
Needs: python -m pip install pypdf"""
import argparse
from pathlib import Path

from pypdf import PdfReader, PdfWriter
from pypdf.errors import PyPdfError


def main(argv=None):
    parser = argparse.ArgumentParser(description="Merge PDF
files.")
    parser.add_argument("output", type=Path)
    parser.add_argument("inputs", type=Path, nargs="+")
    args = parser.parse_args(argv)
    if args.output.exists():
        print(f"Error: {args.output} already exists. Choose a new
name.")
        return 1

    writer = PdfWriter()
    for path in args.inputs:
        try:
            reader = PdfReader(path)
            if reader.is_encrypted:
```

```
                print(f"Skipped {path.name}: password protected")
                continue
            for page in reader.pages:
                writer.add_page(page)
            print(f"Added {path.name}: {len(reader.pages)}
pages")
        except (PyPdfError, OSError) as error:
            print(f"Skipped {path}: {error}")
    if len(writer.pages) == 0:
        print("Nothing was merged.")
        return 1
    with args.output.open("wb") as fh:
        writer.write(fh)
    print(f"Saved {args.output}: {len(writer.pages)} pages")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

**Code explanation:**

- `PdfWriter` collects pages; `add_page` appends one at a time.
- Inputs are processed in the order given on the command line, so you control the order.
- A damaged or encrypted input is skipped with a message while the rest are merged.
- The file is opened in `"wb"` mode because PDFs are binary.

**Sample input:** `sample_data/pdfs/invoice_a.pdf` and `invoice_b.pdf`, one page each.

**Sample output** (real output from my test run, Windows 11, Python 3.14.4, 2026-11-07):

```
Added invoice_a.pdf: 1 pages
Added invoice_b.pdf: 1 pages
Saved invoices_merged.pdf: 2 pages
```

**How to run it** (from the `source` folder):

```
py project16_merge_pdfs.py invoices_merged.pdf
sample_data/pdfs/invoice_a.pdf sample_data/pdfs/invoice_b.pdf
```

**Common errors and fixes:**

- **Nothing was merged.** Every input failed. Check paths and passwords.
- **Error: ... already exists.** Pick a new output name.

**Safe-use notes:** Originals are only read. Bookmarks and form fields of the source PDFs are not handled by this simple script.

**Optional challenge:** Accept a folder and merge all PDFs in alphabetical order.

## Project 17. Splitting a PDF by page range

**Problem:** You need pages 1 to 2 of a handbook in one file and pages 3 to 5 in another.

**Expected result:** One new PDF per range, named with the page numbers.

**Required tools:** Python standard library plus the `pypdf` package.

**Installation:** `py -m pip install pypdf` on Windows (the `py` launcher), `python3 -m pip install pypdf` on macOS and Linux. Run it inside your virtual environment.

**Complete source code** (`project17_split_pdf.py`):

```
"""Project 17: split a PDF by page ranges, for example "1-2,3-5".
Needs: python -m pip install pypdf"""
import argparse
from pathlib import Path

from pypdf import PdfReader, PdfWriter


def parse_ranges(text, page_count):
    """'1-2,5' -> [(1, 2), (5, 5)] with every number checked."""
    ranges = []
    for chunk in text.split(","):
        first, _, last = chunk.strip().partition("-")
        start = int(first)
        end = int(last) if last else start
        if not 1 <= start <= end <= page_count:
            raise ValueError(f"range '{chunk}' is outside 1-
{page_count}")
        ranges.append((start, end))
    return ranges


def main(argv=None):
    parser = argparse.ArgumentParser(description="Split a PDF.")
    parser.add_argument("pdf", type=Path)
    parser.add_argument("outdir", type=Path)
```

```
    parser.add_argument("--ranges", required=True, help='e.g. "1-
2,3-5"')
    args = parser.parse_args(argv)
    if not args.pdf.is_file():
        print(f"Error: {args.pdf} not found.")
        return 1
    reader = PdfReader(args.pdf)
    try:
        ranges = parse_ranges(args.ranges, len(reader.pages))
    except ValueError as error:
        print(f"Error: {error}")
        return 1

    args.outdir.mkdir(parents=True, exist_ok=True)
    for start, end in ranges:
        target = args.outdir / f"{args.pdf.stem}_pages_{start}-
{end}.pdf"
        if target.exists():
            print(f"Skipped {target.name}: already exists")
            continue
        writer = PdfWriter()
        for index in range(start - 1, end):  # pypdf pages start
at 0
            writer.add_page(reader.pages[index])
        with target.open("wb") as fh:
            writer.write(fh)
        print(f"Wrote {target.name} ({end - start + 1} pages)")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

**Code explanation:**

- `parse_ranges` turns "1-2,3-5" into pairs and checks each against the page count before anything is written.
- Humans count pages from 1, pypdf from 0, so the loop uses `range(start - 1, end)`.
- An existing target is skipped, not replaced.
- A bad range such as 4-9 stops the run with a clear message.

**Sample input:** `sample_data/pdfs/handbook.pdf` (5 pages).

**Sample output** (real output from my test run, Windows 11, Python 3.14.4, 2026-11-07):

Valid ranges:

```
Wrote handbook_pages_1-2.pdf (2 pages)
Wrote handbook_pages_3-5.pdf (3 pages)
```

Range outside the document:

```
Error: range '4-9' is outside 1-5
```

**How to run it** (from the `source` folder):

```
py project17_split_pdf.py sample_data/pdfs/handbook.pdf
handbook_parts --ranges 1-2,3-5
py project17_split_pdf.py sample_data/pdfs/handbook.pdf
handbook_parts --ranges 4-9
```

**Common errors and fixes:**

- **ValueError: invalid literal for int().** A range contained text. Use digits, a dash and commas only.
- **Skipped ... already exists.** Delete or rename the old file yourself, then run again.

**Safe-use notes:** The source PDF is never modified. Check that each part opens in a PDF viewer.

**Optional challenge:** Add --each to write one file per page.

CHAPTER 8

# Text processing

Plain text is the easiest data to process, and the most useful skills here are the regular expression and the `Counter`. A regular expression is a compact pattern for text. Useful pieces:

| Pattern | Meaning |
|---|---|
| `\d` | one digit |
| `\w+` | one or more letters, digits or underscores |
| `^` and `$` | start and end of a line |
| `(...)` | a group you can pull out |
| `[-*]` | one of the listed characters |

Build patterns step by step and test them on small samples. The documentation for the `re` module at docs.python.org is the reference.

## Project 18. Word frequency counter

**Problem:** You want to know which words dominate a set of notes or articles.

**Expected result:** A ranked list of the most common words, optionally saved as CSV.

**Required tools:** Python standard library only.

**Installation:** nothing to install. Use `py` on Windows or `python3` on macOS and Linux to run it.

**Complete source code** (`project18_word_frequency.py`):

```
"""Project 18: count the most common words in all .txt and .md
files of a
folder. Read only."""
import argparse
import csv
import re
from collections import Counter
from pathlib import Path

# A short list on purpose; add words that clutter your results.
STOP_WORDS = {"the", "a", "an", "and", "or", "is", "are", "was",
"to", "of",
              "in", "on", "it", "be", "for", "with", "when",
"this", "that"}


def main(argv=None):
    parser = argparse.ArgumentParser(description="Word frequency
counter.")
    parser.add_argument("folder", type=Path)
    parser.add_argument("--top", type=int, default=10)
    parser.add_argument("--csv", type=Path, help="also save the
table here")
    args = parser.parse_args(argv)
    if not args.folder.is_dir():
        print(f"Error: {args.folder} is not a folder.")
        return 1
```

```
    counts, files = Counter(), 0
    for path in sorted(args.folder.rglob("*")):
        if path.suffix.lower() not in {".txt", ".md"}:
            continue
        try:
            text = path.read_text(encoding="utf-8")
        except (UnicodeDecodeError, OSError) as error:
            print(f"Skipped {path.name}: {error}")
            continue
        files += 1
        # \w+ keeps letters and digits; apostrophes split words,
a known limit.
        words = re.findall(r"\w+", text.lower())
        counts.update(w for w in words if w not in STOP_WORDS and
len(w) > 1)

    print(f"Read {files} files, {sum(counts.values())} counted
words, "
          f"{len(counts)} different words.")
    for word, number in counts.most_common(args.top):
        print(f"  {word:<12} {number}")
    if args.csv:
        with args.csv.open("w", newline="", encoding="utf-8") as
fh:
            writer = csv.writer(fh)
            writer.writerow(["word", "count"])
            writer.writerows(counts.most_common())
        print(f"Saved {args.csv}")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

**Code explanation:**

- `re.findall(r"\w+", text.lower())` splits text into lowercase words.
- `Counter` counts them; `most_common(n)` returns the top n.
- STOP_WORDS removes very common words that carry no meaning. The list is short on purpose; extend it for your texts.
- Files that are not valid UTF-8 are reported and skipped.

**Sample input:** `sample_data/notes` with `a.txt`, `b.txt` and `c.md` (short English texts).

**Sample output** (real output from my test run, Windows 11, Python 3.14.4, 2026-11-07):

```
Read 3 files, 33 counted words, 26 different words.
  tools        4
  acme         3
  python       2
  task         2
  makes        1
  small        1
```

**How to run it** (from the `source` folder):

```
py project18_word_frequency.py sample_data/notes --top 6
```

**Common errors and fixes:**

- **Skipped ...: 'utf-8' codec can't decode.** Save the file as UTF-8 or change the encoding in read_text.
- **Words like don and t appear.** The pattern splits at apostrophes. Use r"[\w']+" to keep them.

**Safe-use notes:** Read only. For private documents keep the output file private too.

**Optional challenge:** Count two-word phrases (bigrams) instead of single words.

## Project 19. Find and replace across files

**Problem:** A product name changed and appears in dozens of text files.

**Expected result:** A list of files and match counts; with --apply the text is replaced and each changed file gets a .bak copy.

**Required tools:** Python standard library only.

**Installation:** nothing to install. Use `py` on Windows or `python3` on macOS and Linux to run it.

**Complete source code** (`project19_find_replace.py`):

```
"""Project 19: replace text in every .txt and .md file under a
folder.
Dry run by default. With --apply each changed file is first
copied to
NAME.bak so you can restore it."""
import argparse
import shutil
from pathlib import Path


def main(argv=None):
    parser = argparse.ArgumentParser(description="Find and
replace in files.")
    parser.add_argument("folder", type=Path)
    parser.add_argument("find")
    parser.add_argument("replace")
    parser.add_argument("--apply", action="store_true")
    args = parser.parse_args(argv)
    if not args.folder.is_dir():
        print(f"Error: {args.folder} is not a folder.")
        return 1
    if not args.find:
        print("Error: the text to find cannot be empty.")
        return 1

    changed = replacements = 0
    for path in sorted(args.folder.rglob("*")):
```

```
        if path.suffix.lower() not in {".txt", ".md"}:
            continue
        try:
            text = path.read_text(encoding="utf-8")
        except (UnicodeDecodeError, OSError) as error:
            print(f"Skipped {path.name}: {error}")
            continue
        found = text.count(args.find)
        if not found:
            continue
        changed += 1
        replacements += found
        print(f"{path.relative_to(args.folder)}: {found}
match(es)")
        if args.apply:
            backup = path.with_name(path.name + ".bak")
            if backup.exists():
                print(f"  Stopped for this file: {backup.name}
already exists")
                continue
            shutil.copy2(path, backup)  # backup BEFORE the
change
            path.write_text(text.replace(args.find,
args.replace),
                            encoding="utf-8")
    mode = "Replaced" if args.apply else "Dry run, would replace"
    print(f"{mode} {replacements} matches in {changed} files.")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

**Code explanation:**

- The default run only counts matches, so you see the scope before any change.
- `shutil.copy2` creates `NAME.bak` before the file is rewritten. If a .bak already exists the script refuses to touch that file.
- Only `.txt` and `.md` files are processed, to avoid corrupting binary files.

- The replacement is case sensitive. `ACME Tools` does not match `acme tools`.

**Sample input:** `sample_data/notes`: the file `c.md` contains "ACME Tools" three times.

**Sample output** (real output from my test run, Windows 11, Python 3.14.4, 2026-11-07):

Dry run:

```
c.md: 3 match(es)
Dry run, would replace 3 matches in 1 files.
```

Real run:

```
c.md: 3 match(es)
Replaced 3 matches in 1 files.
```

Dry run again (nothing left to replace):

```
Dry run, would replace 0 matches in 0 files.
```

**How to run it** (from the `source` folder):

```
py project19_find_replace.py sample_data/notes "ACME Tools"
"Northwind Tools"
py project19_find_replace.py sample_data/notes "ACME Tools"
"Northwind Tools" --apply
py project19_find_replace.py sample_data/notes "ACME Tools"
"Northwind Tools"
```

These commands change or create files. To repeat a test, delete the `sample_data` folder and run `py make_sample_data.py` again.

**Common errors and fixes:**

- **Stopped for this file: c.md.bak already exists.** Move or rename the old backup. The script will not overwrite it.
- **Dry run, would replace 0 matches.** Check spelling and capitalisation of the search text.

**Safe-use notes:** Always dry run first, and keep the .bak files until you have checked the results. Version control (Git) is even better for text projects.

**Optional challenge:** Add a --ignore-case option using re.sub with re.IGNORECASE.

## Project 20. Release notes text cleaner

**Problem:** Release notes were typed quickly: mixed bullet symbols, extra spaces, blank lines, repeated items.

**Expected result:** A tidy file where each item is one bullet, starts with a capital, has no trailing full stop and appears once.

**Required tools:** Python standard library only.

**Installation:** nothing to install. Use `py` on Windows or `python3` on macOS and Linux to run it.

**Complete source code** (`project20_release_notes_cleaner.py`):

```
"""Project 20: tidy a rough list of release notes into consistent
bullets.
The original file is never changed."""
import argparse
import re
from pathlib import Path

BULLET = re.compile(r"^\s*[-*•]\s*")  # -, * or the bullet
character


def clean_lines(text):
    seen, result = set(), []
    for raw in text.splitlines():
        if not BULLET.match(raw) and not raw.strip():
            continue  # drop blank lines
        line = BULLET.sub("", raw)
        line = " ".join(line.split())  # collapse runs of spaces,
trim ends
        if not line:
            continue
        # Capitalise the first word; tone down a shouting ALL
CAPS first word.
        first, _, rest = line.partition(" ")
        first = first.capitalize() if first.isupper() else
first[0].upper() + first[1:]
        line = f"{first} {rest}".strip().rstrip(".")
```

```
        if line.lower() in seen:  # case-insensitive duplicate
            continue
        seen.add(line.lower())
        result.append(line)
    return result


def main(argv=None):
    parser = argparse.ArgumentParser(description="Clean release
notes.")
    parser.add_argument("input", type=Path)
    parser.add_argument("output", type=Path)
    args = parser.parse_args(argv)
    if not args.input.is_file():
        print(f"Error: {args.input} not found.")
        return 1
    if args.output.exists():
        print(f"Error: {args.output} already exists. Choose a new
name.")
        return 1

    original = args.input.read_text(encoding="utf-8")
    lines = clean_lines(original)
    args.output.write_text("\n".join(f"- {l}" for l in lines) +
"\n",
                           encoding="utf-8")
    print(f"{len(original.splitlines())} lines in, {len(lines)}
bullets out:")
    print(args.output.read_text(encoding="utf-8"), end="")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

**Code explanation:**

- The `BULLET` pattern matches -, * or the bullet character at the start of a line, so one regular expression handles all three.
- `" ".join(line.split())` collapses repeated spaces.
- The first word is capitalised; an ALL CAPS first word is toned down to a normal capital.

- `seen` holds lowercase versions of finished lines, so Added dark mode and -added dark mode count as one.

**Sample input:** `sample_data/release_notes_raw.txt`, 9 lines with mixed bullets, spaces and a repeated item.

**Sample output** (real output from my test run, Windows 11, Python 3.14.4, 2026-11-07):

```
9 lines in, 5 bullets out:
- Fixed login bug
- Added dark mode
- Improved search speed
- Fixed crash on startup
- Update docs
```

**How to run it** (from the `source` folder):

```
py project20_release_notes_cleaner.py
sample_data/release_notes_raw.txt release_notes_clean.txt
```

**Common errors and fixes:**

- **Error: ... already exists.** Choose a new output file name.
- **A proper name was lowercased.** Only an ALL CAPS first word is changed. Add exceptions for product names in clean_lines.

**Safe-use notes:** The input is only read. Rules like removing full stops are style choices; change them to match your team.

**Optional challenge:** Group lines by their first word (Fixed, Added, Improved) under headings.

## Project 21. Log file summary

**Problem:** A log file with thousands of lines is hard to scan. You want counts and the most common errors.

**Expected result:** Counts per level, the three most common errors, the busiest hour and the number of lines that could not be read.

**Required tools:** Python standard library only.

**Installation:** nothing to install. Use `py` on Windows or `python3` on macOS and Linux to run it.

**Complete source code** (`project21_log_summary.py`):

```
"""Project 21: summarise a log file: counts per level, the most
common error
messages, and busy hours. Read only."""
import argparse
import re
from collections import Counter
from pathlib import Path

# Example line: 2026-11-01 08:35:00 ERROR payment timeout
LINE = re.compile(r"^(\d{4}-\d{2}-\d{2}) (\d{2}):\d{2}:\d{2} "
                  r"(DEBUG|INFO|WARNING|ERROR|CRITICAL) (.*)$")


def main(argv=None):
    parser = argparse.ArgumentParser(description="Log file
summary.")
    parser.add_argument("logfile", type=Path)
    args = parser.parse_args(argv)
    if not args.logfile.is_file():
        print(f"Error: {args.logfile} not found.")
        return 1

    levels, errors, hours = Counter(), Counter(), Counter()
    unreadable = 0
    # errors="replace": one bad byte should not stop the whole
summary.
    with args.logfile.open(encoding="utf-8", errors="replace") as
```

```
fh:
        for line in fh:
            match = LINE.match(line.strip())
            if not match:
                unreadable += 1
                continue
            day, hour, level, message = match.groups()
            levels[level] += 1
            hours[f"{day} {hour}:00"] += 1
            if level in ("ERROR", "CRITICAL"):
                errors[message] += 1

    print(f"Lines understood: {sum(levels.values())}, not
understood: {unreadable}")
    for level, number in levels.most_common():
        print(f"  {level:<9} {number}")
    print("Most common errors:")
    for message, number in errors.most_common(3):
        print(f"  {number} x {message}")
    busiest, count = hours.most_common(1)[0] if hours else
("none", 0)
    print(f"Busiest hour: {busiest} with {count} entries")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

**Code explanation:**

- The `LINE` regular expression names the parts of a line: date, hour, level and message. `groups()` returns them in that order.
- Lines that do not match are counted as not understood, not hidden.
- `errors="replace"` swaps undecodable bytes for a placeholder, so one bad byte does not stop the run.
- `Counter.most_common` does the ranking.

**Sample input:** `sample_data/app.log`: 41 lines, one of them deliberately not a log entry. The format is `2026-11-01 08:35:00 ERROR payment timeout`.

**Sample output** (real output from my test run, Windows 11, Python 3.14.4, 2026-11-07):

```
Lines understood: 40, not understood: 1
  INFO      30
  ERROR     7
  WARNING   3
Most common errors:
  6 x db connection lost
  1 x payment timeout
Busiest hour: 2026-11-01 08:00 with 6 entries
```

**How to run it** (from the `source` folder):

```
py project21_log_summary.py sample_data/app.log
```

**Common errors and fixes:**

- **Lines understood: 0.** Your log format differs. Adapt the LINE pattern.
- **IndexError on empty logs.** The script handles it: busiest hour shows none.

**Safe-use notes:** Logs can hold personal data or secrets. Do not paste a real log into shared places.

**Optional challenge:** Add --since HH:MM to look only at a time window.

CHAPTER 9

# Reports and spreadsheets

A report is a repeatable summary. The reliable approach has three steps: load the data, compute the numbers once, then write all outputs from those numbers. That is how Project 22 keeps its Excel, Markdown and HTML files consistent.

Choose the output for the reader. Excel suits people who will filter or calculate further. Markdown suits notes and version control. HTML opens in any browser and can be emailed as an attachment. Do not email or publish a report that contains personal data without checking who is allowed to see it.

## Project 22. Monthly report generation

**Problem:** Every month you build the same sales summary by hand and compare it with the month before.

**Expected result:** For a chosen month: an Excel file, a Markdown file and an HTML file with units and revenue per region and the percentage change on the previous month.

**Required tools:** Python standard library plus the `openpyxl` package.

**Installation:** `py -m pip install openpyxl` on Windows (the `py` launcher), `python3 -m pip install openpyxl` on macOS and Linux. Run it inside your virtual environment.

**Complete source code** (`project22_monthly_report.py`):

```
"""Project 22: build a monthly sales report (Excel, Markdown and
HTML) from a
folder of sales_YYYY_MM.csv files, with a comparison to the
previous month.
Needs: python -m pip install openpyxl"""
import argparse
import csv
import html
from collections import defaultdict
from pathlib import Path

from openpyxl import Workbook
from openpyxl.styles import Font


def load_month(folder, month):
    """Return {region: [units, revenue]} for month like '2026-
10'."""
    path = folder / f"sales_{month.replace('-', '_')}.csv"
    if not path.is_file():
        return None
    totals = defaultdict(lambda: [0, 0.0])
    with path.open(newline="", encoding="utf-8-sig") as fh:
        for row in csv.DictReader(fh):
```

```
            units = int(row["units"])
            totals[row["region"]][0] += units
            totals[row["region"]][1] += units *
float(row["unit_price"])
    return dict(totals)


def previous(month):
    year, mon = map(int, month.split("-"))
    return f"{year - 1}-12" if mon == 1 else f"{year}-{mon -
1:02d}"


def main(argv=None):
    parser = argparse.ArgumentParser(description="Monthly sales
report.")
    parser.add_argument("folder", type=Path)
    parser.add_argument("--month", required=True, help="YYYY-MM")
    parser.add_argument("--outdir", type=Path,
default=Path("report_out"))
    args = parser.parse_args(argv)

    current = load_month(args.folder, args.month)
    if current is None:
        print(f"Error: no sales file for {args.month} in
{args.folder}")
        return 1
    before = load_month(args.folder, previous(args.month)) or {}
    args.outdir.mkdir(parents=True, exist_ok=True)

    rows = []
    for region in sorted(current):
        units, revenue = current[region]
        old = before.get(region, [0, 0.0])[1]
        change = f"{(revenue - old) / old * 100:+.1f}%" if old
else "n/a"
        rows.append((region, units, round(revenue, 2), change))
    total_units = sum(r[1] for r in rows)
    total_rev = round(sum(r[2] for r in rows), 2)
    heading = ["Region", "Units", "Revenue", "vs previous month"]

    wb = Workbook()
    ws = wb.active
    ws.title = args.month
    ws.append(heading)
    for r in rows:
```

```
        ws.append(list(r))
    ws.append(["Total", total_units, total_rev, ""])
    for cell in ws[1] + ws[ws.max_row]:
        cell.font = Font(bold=True)
    wb.save(args.outdir / f"report_{args.month}.xlsx")

    md = [f"# Sales report {args.month}", "",
          "| " + " | ".join(heading) + " |", "|---|---:|---:|---:|"]
    md += [f"| {r[0]} | {r[1]} | {r[2]:.2f} | {r[3]} |" for r in rows]
    md += [f"| **Total** | **{total_units}** | **{total_rev:.2f}** | |"]
    (args.outdir / f"report_{args.month}.md").write_text("\n".join(md) + "\n",
                                                         encoding="utf-8")

    cells = "".join("<tr>" + "".join(f"<td>{html.escape(str(c))}</td>" for c in r)
                    + "</tr>" for r in rows)
    page = (f"<!doctype html><meta charset='utf-8'><title>Sales {args.month}"
            f"</title><h1>Sales report {args.month}</h1><table border='1' "
            f"cellpadding='6'><tr>{''.join(f'<th>{h}</th>' for h in heading)}"
            f"</tr>{cells}<tr><th>Total</th><th>{total_units}</th>"
            f"<th>{total_rev:.2f}</th><th></th></tr></table>")
    (args.outdir / f"report_{args.month}.html").write_text(page, encoding="utf-8")

    print("\n".join(md))
    print(f"\nFiles written to {args.outdir}: xlsx, md, html")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

**Code explanation:**

- `load_month` finds `sales_YYYY_MM.csv`, adds up units and revenue per region and returns None if the file is missing.
- `previous` handles January by returning December of the year before.
- The change is `(revenue - old) / old * 100`. When last month had no data, the script prints n/a rather than dividing by zero.
- `html.escape` makes sure text from the data cannot break the HTML page.
- One set of numbers feeds all three outputs, so they cannot disagree.

**Sample input:** `sample_data/sales/sales_2026_09.csv` and `sales_2026_10.csv`, each 24 invented rows. The percentages are large because the sample data is random.

**Sample output** (real output from my test run, Windows 11, Python 3.14.4, 2026-11-07):

```
# Sales report 2026-10

| Region | Units | Revenue | vs previous month |
|---|---:|---:|---:|
| East | 182 | 1267.80 | +84.1% |
| North | 116 | 753.95 | +107.4% |
| South | 89 | 671.20 | -58.4% |
| **Total** | **387** | **2692.95** | |

Files written to report_out: xlsx, md, html
```

**How to run it** (from the `source` folder):

```
py project22_monthly_report.py sample_data/sales --month 2026-10
```

**Common errors and fixes:**

- **Error: no sales file for 2026-10.** File names must follow sales_YYYY_MM.csv.

- **n/a in the change column.** The previous month has no data for that region.

**Safe-use notes:** Writes only into the output folder and overwrites earlier reports for the same month. Rename or move a report you want to keep before re-running. Check totals against the source at least once by hand.

**Optional challenge:** Add a bar chart image or an Excel chart for the three regions.

## Project 23. Email preparation drafts

**Problem:** Payment reminders follow one template. You want drafts ready to review, not messages sent automatically.

**Expected result:** One .eml file per valid recipient. Rows with a bad address are skipped. Nothing is sent.

**Required tools:** Python standard library only.

**Installation:** nothing to install. Use `py` on Windows or `python3` on macOS and Linux to run it.

**Complete source code** (`project23_email_drafts.py`):

```
"""Project 23: prepare payment reminder emails as .eml files from
a CSV.
Nothing is sent: the script has no mail server code and no
password.
You open each file in your mail program, read it, and send it
yourself."""
import argparse
import csv
import re
from email.message import EmailMessage
from pathlib import Path

EMAIL = re.compile(r"^[^@\s]+@[^@\s]+\.[^@\s]+$")
TEMPLATE = """Hello {name},

This is a friendly reminder that invoice {invoice} for {amount}
is due on
{due}. If you have already paid, please ignore this message.

Kind regards
"""


def main(argv=None):
    parser = argparse.ArgumentParser(description="Prepare email
drafts.")
    parser.add_argument("input", type=Path)
```

```
    parser.add_argument("outdir", type=Path)
    parser.add_argument("--sender", required=True, help="your
address")
    args = parser.parse_args(argv)
    if not args.input.is_file():
        print(f"Error: {args.input} not found.")
        return 1
    if not EMAIL.match(args.sender):
        print("Error: --sender does not look like an email
address.")
        return 1
    args.outdir.mkdir(parents=True, exist_ok=True)

    made = skipped = 0
    with args.input.open(newline="", encoding="utf-8-sig") as fh:
        for line_no, row in enumerate(csv.DictReader(fh),
start=2):
            if not EMAIL.match(row["email"].strip()):
                print(f"Skipped line {line_no}: bad address
'{row['email']}'")
                skipped += 1
                continue
            msg = EmailMessage()
            msg["From"] = args.sender
            msg["To"] = row["email"].strip()
            msg["Subject"] = f"Reminder: invoice
{row['invoice']}"
            msg.set_content(TEMPLATE.format(
                name=row["name"], invoice=row["invoice"],
                amount=f"{float(row['amount']):.2f}",
due=row["due"]))
            target = args.outdir / f"{row['invoice']}.eml"
            if target.exists():
                print(f"Skipped {target.name}: already exists")
                skipped += 1
                continue
            target.write_bytes(bytes(msg))  # a file on disk,
nothing sent
            print(f"Draft written: {target.name} (to
{msg['To']})")
            made += 1
    print(f"{made} drafts prepared, {skipped} skipped. Nothing
was sent.")
    return 0
```

```
if __name__ == "__main__":
    raise SystemExit(main())
```

**Code explanation:**

- `EmailMessage` builds a standards-based message object; no mail server is involved. The script imports no sending code and asks for no password.
- `TEMPLATE.format(...)` fills the placeholders from each CSV row.
- Addresses are checked with a simple pattern first. It catches typos like a missing @, not wrong mailboxes.
- `bytes(msg)` produces the raw message, saved as an .eml file you can open in a mail program.

**Sample input:** `sample_data/reminders.csv` with 3 rows; the third has an invalid address. Addresses use example.com, a domain reserved for examples.

**Sample output** (real output from my test run, Windows 11, Python 3.14.4, 2026-11-07):

```
Draft written: INV-2026-0001.eml (to alice@example.com)
Draft written: INV-2026-0002.eml (to bob@example.com)
Skipped line 4: bad address 'missing-at-sign'
2 drafts prepared, 1 skipped. Nothing was sent.
```

**How to run it** (from the `source` folder):

```
py project23_email_drafts.py sample_data/reminders.csv drafts --
sender me@example.com
```

**Common errors and fixes:**

- **Skipped line N: bad address.** Fix the address in the CSV.
- **The .eml file does not open as a draft.** Mail programs treat .eml files differently. I did not test any mail program for this book; open the file and copy the text if needed.

**Safe-use notes:** Review each draft before you send it yourself. Do not add a password or SMTP login to a script. Sending bulk mail is covered by privacy and anti-spam rules in many countries; check the rules for your use.

**Optional challenge:** Write a plain .txt copy of each message too, for people who prefer copy and paste.

CHAPTER 10

# APIs and responsible web data usage

An API (application programming interface) is a way for programs to request data. A web API usually returns JSON, which Python reads with the `json` module into dictionaries and lists.

## Rules for using web data

- **Prefer an official API** to scraping pages.
- **Read the terms of use.** They decide what you may do with the data, whatever the technology allows.
- **Check robots.txt.** RFC 9309 describes it as a way for site owners to ask crawlers to avoid certain paths, and states that its rules are not a form of access authorisation. It is a courtesy signal, not permission or protection. Terms and law still apply.
- **Do not scrape sites that forbid it,** and do not bypass logins, paywalls or limits.
- **Limit your rate.** Pause between requests, set a timeout, and cache results so you do not ask twice.
- **Never put credentials in code.** Read them from environment variables, and never save them in files you share or upload.

Setting an environment variable for one terminal session:

```
# Windows PowerShell
$env:EXAMPLE_API_TOKEN = "paste-token-here"
```

```
# macOS and Linux
export EXAMPLE_API_TOKEN="paste-token-here"
```

In Python, `os.environ.get("EXAMPLE_API_TOKEN")` reads it. The test API in Project 24, JSONPlaceholder, needs no key. On 2026-11-07 its home page described it as free, with no signup, for tests and tutorials, and listed 10 users and 200 to-dos among its resources. It states no rate limit, so the project stays polite anyway. Its data is fake.

## Project 24. API data retrieval

**Problem:** You want to pull data from a public web service and summarise it, without hammering the service or storing keys in code.

**Expected result:** A table of users and their completed to-do counts from the JSONPlaceholder test API, saved to a local cache file. If the network fails, the script says so and shows the cached copy.

**Required tools:** Python standard library only.

**Installation:** nothing to install. Use `py` on Windows or `python3` on macOS and Linux to run it.

**Complete source code** (`project24_api_retrieval.py`):

```
"""Project 24: fetch public test data from JSONPlaceholder (a
free,
documented, no-key API meant for practice) and summarise it.
Uses only the standard library. Be polite: few requests, a pause
between
them, a timeout, and a local cache file so reruns need no
network."""
import argparse
import json
import os
import time
import urllib.error
import urllib.request
from pathlib import Path

BASE_URL = "https://jsonplaceholder.typicode.com"


def fetch_json(url, token=None):
    headers = {"User-Agent": "python-automation-book-demo/1.0",
               "Accept": "application/json"}
    if token:  # a real API key would come from an environment
variable
        headers["Authorization"] = f"Bearer {token}"
    request = urllib.request.Request(url, headers=headers)
    with urllib.request.urlopen(request, timeout=10) as response:
```

```
        return json.loads(response.read().decode("utf-8"))


def main(argv=None):
    parser = argparse.ArgumentParser(description="API data
retrieval.")
    parser.add_argument("--users", type=int, default=3, help="how
many users")
    parser.add_argument("--base-url", default=BASE_URL)
    parser.add_argument("--cache", type=Path,
default=Path("api_cache.json"))
    args = parser.parse_args(argv)
    if not 1 <= args.users <= 10:
        print("Error: --users must be between 1 and 10.")
        return 1
    token = os.environ.get("EXAMPLE_API_TOKEN")  # optional;
never hard-code

    try:
        users = fetch_json(f"{args.base_url}/users", token)[:
args.users]
        report = []
        for user in users:
            time.sleep(1)  # rate limiting: one request per
second
            todos = fetch_json(f"{args.base_url}/todos?userId=
{user['id']}", token)
            done = sum(1 for t in todos if t["completed"])
            report.append({"id": user["id"], "name":
user["name"],
                           "city": user["address"]["city"],
                           "todos": len(todos), "done": done})
        args.cache.write_text(json.dumps(report, indent=2),
encoding="utf-8")
        source = "live API"
    except urllib.error.HTTPError as error:
        print(f"The server answered with an error: {error.code}
{error.reason}")
        return 1
    except (urllib.error.URLError, TimeoutError) as error:
        print(f"Could not reach the API ({error}).")
        if not args.cache.exists():
            print("No cached copy exists yet. Try again when
online.")
            return 1
        report = json.loads(args.cache.read_text(encoding="utf-
```

```
8"))
        source = f"cache file {args.cache.name} (may be out of
date)"
    except (json.JSONDecodeError, KeyError) as error:
        print(f"Unexpected data from the API: {error!r}")
        return 1

    print(f"Data source: {source}")
    for r in report:
        print(f"  {r['id']}. {r['name']:<22} {r['city']:<14} "
              f"{r['done']}/{r['todos']} todos done")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

**Code explanation:**

- `urllib.request` is part of the standard library, so there is nothing to install.
- A `User-Agent` header identifies the script. `timeout=10` stops the script waiting forever.
- `time.sleep(1)` leaves a pause between requests. The API page states no limits, so the script stays polite anyway.
- `EXAMPLE_API_TOKEN` shows how a key would be read from an environment variable. JSONPlaceholder needs no key, so the variable is optional.
- The `except` blocks separate three cases: the server answered with an error, the network is unreachable, and the data has an unexpected shape.

**Sample input:** No input file. The service is `https://jsonplaceholder.typicode.com`, a free fake REST API for tests (checked 2026-11-07: no key, no signup). Its data is made up.

**Sample output** (real output from my test run, Windows 11, Python 3.14.4, 2026-11-07):

Live API (it was reachable when I tested):

```
Data source: live API
  1. Leanne Graham          Gwenborough    11/20 todos done
  2. Ervin Howell           Wisokyburgh    8/20 todos done
  3. Clementine Bauch       McKenziehaven  7/20 todos done
```

Unreachable server (simulated with http://localhost:9):

```
Could not reach the API (<urlopen error [WinError 10061] No
connection could be made because the target machine actively
refused it>).
Data source: cache file api_cache.json (may be out of date)
  1. Leanne Graham          Gwenborough    11/20 todos done
  2. Ervin Howell           Wisokyburgh    8/20 todos done
  3. Clementine Bauch       McKenziehaven  7/20 todos done
```

**How to run it** (from the `source` folder):

```
py project24_api_retrieval.py --users 3
py project24_api_retrieval.py --base-url http://localhost:9
```

**Common errors and fixes:**

- **Could not reach the API.** Check your connection, proxy or firewall. The cached copy is used if it exists.
- **HTTP Error 429.** You are sending too many requests. Wait, and raise the sleep time.

**Safe-use notes:** Read the terms and robots.txt of any site before you automate it. robots.txt asks crawlers to stay away from some paths but is not access authorisation; terms of use still apply. Do not scrape sites that forbid it, and prefer an official API.

**Optional challenge:** Save results as CSV and add a --refresh-after-hours option that avoids calls when the cache is recent.

CHAPTER 11

# Scheduling

A script that runs by itself needs three things: a trigger, a reliable way to start, and a record of what happened.

## Make the script scheduler-friendly

- Use absolute paths for Python, the script and the folders. A scheduler does not start in your folder.
- Return a non-zero exit code on failure.
- Log to a file (Project 25 shows how).
- Make a repeat run harmless. Project 25 does nothing if today's report exists.
- Use the virtual environment's Python by its full path, so the right packages are found.

## Logging

```
import logging
logging.basicConfig(filename="job.log", level=logging.INFO,
                    format="%(asctime)s %(levelname)s %
(message)s")
logging.info("Job started")
try:
    1 / 0
except ZeroDivisionError:
    logging.exception("Job failed")   # writes the full traceback
```

Levels from low to high: DEBUG, INFO, WARNING, ERROR, CRITICAL. Use `print` for what the user should see; use `logging` for the record.

## Operating system schedulers

I did not test any of the following. Treat them as starting points and test with a harmless script (one that only writes a line to a log) first.

**Windows Task Scheduler.** Microsoft describes it as a component that runs tasks when triggers such as a time, logon or boot occur. You can use the Task Scheduler app (Create Basic Task) or the command line. Example, daily at 07:30:

```
schtasks /create /tn "DailyReport" /sc daily /st 07:30 ^
  /tr "\"C:\proj\.venv\Scripts\python.exe\"
\"C:\proj\project25_scheduled_report.py\" C:\proj\inbox
C:\proj\reports"
```

**cron (Linux, macOS).** Edit your table with `crontab -e`. The five time fields are minute, hour, day of month, month, day of week.

```
30 7 * * 1-5 /home/me/proj/.venv/bin/python
/home/me/proj/project25_scheduled_report.py /home/me/proj/inbox
/home/me/proj/reports >> /home/me/proj/cron.out 2>&1
```

That line means 07:30 on weekdays, with output appended to a file.

**launchd (macOS).** A property list file in `~/Library/LaunchAgents/` names the program and a `StartCalendarInterval`. It is loaded with `launchctl`. The details differ between macOS versions; use Apple's current documentation for your version.

Remember that a laptop that is asleep or off does not run its jobs on time, and an unattended job runs with your permissions.

## Python-only options

The simplest is a loop with a pause, as Project 25's `--every-seconds` and `--runs` options do. It runs only while the program is open and stops when you log off. The standard library also has a `sched` module for in-process timers. Third-party schedulers exist; I did not test any.

## Project 25. Scheduled report preparation

**Problem:** A report should be prepared every day by a scheduler, leave a log, and not repeat itself if it runs twice.

**Expected result:** A dated Markdown report built from the newest CSV in an inbox folder, a rotating log file, and a clean exit code.

**Required tools:** Python standard library only.

**Installation:** nothing to install. Use `py` on Windows or `python3` on macOS and Linux to run it.

**Complete source code** (`project25_scheduled_report.py`):

```
"""Project 25: a report job that is safe to run from a scheduler.
It takes
the newest CSV in an inbox folder, writes a dated Markdown
report, logs
everything to a file, and skips the run if today's report already
exists.
Python-only repeat: add --every-seconds N --runs M (see the
book)."""
import argparse
import csv
import logging
import time
from collections import Counter
from datetime import date
from decimal import Decimal
from logging.handlers import RotatingFileHandler
from pathlib import Path


def run_once(inbox, outdir, log):
    files = sorted(inbox.glob("*.csv"), key=lambda p:
p.stat().st_mtime)
    if not files:
        log.warning("No CSV files in %s", inbox)
        return 1
    newest = files[-1]
    target = outdir / f"report_{date.today().isoformat()}.md"
```

```
    if target.exists():
        log.info("Report %s already exists: nothing to do",
target.name)
        return 0
    totals = Counter()
    with newest.open(newline="", encoding="utf-8-sig") as fh:
        for row in csv.DictReader(fh):
            totals[row["category"]] += Decimal(row["amount"])
    lines = [f"# Spending report {date.today().isoformat()}", "",
             f"Source: {newest.name}", ""]
    lines += [f"- {cat}: {amount:.2f}" for cat, amount in
totals.most_common()]
    lines += ["", f"Total: {sum(totals.values()):.2f}"]
    outdir.mkdir(parents=True, exist_ok=True)
    target.write_text("\n".join(lines) + "\n", encoding="utf-8")
    log.info("Wrote %s from %s (%d categories)", target.name,
newest.name,
             len(totals))
    return 0


def main(argv=None):
    parser = argparse.ArgumentParser(description="Scheduled
report job.")
    parser.add_argument("inbox", type=Path)
    parser.add_argument("outdir", type=Path)
    parser.add_argument("--log", type=Path,
default=Path("report_job.log"))
    parser.add_argument("--every-seconds", type=int, default=0)
    parser.add_argument("--runs", type=int, default=1)
    args = parser.parse_args(argv)

    log = logging.getLogger("report_job")
    log.setLevel(logging.INFO)
    # A rotating file keeps the log from growing without limit.
    handler = RotatingFileHandler(args.log, maxBytes=200_000,
backupCount=3,
                                  encoding="utf-8")
    handler.setFormatter(logging.Formatter("%(asctime)s %
(levelname)s %(message)s"))
    log.addHandler(handler)
    log.addHandler(logging.StreamHandler())  # also show on
screen

    if not args.inbox.is_dir():
        log.error("Inbox %s is not a folder", args.inbox)
```

```
        return 1
    status = 0
    for run in range(args.runs):
        try:
            status = run_once(args.inbox, args.outdir, log)
        except Exception:  # a scheduled job must log the reason
and not vanish
            log.exception("Report job failed")
            status = 1
        if run < args.runs - 1:
            time.sleep(args.every_seconds)
    return status


if __name__ == "__main__":
    raise SystemExit(main())
```

**Code explanation:**

- `run_once` is the whole job. Keeping it in a function lets you call it once, in a loop, or from a scheduler.
- If today's report exists, the job logs it and stops. That makes accidental double runs harmless (idempotent).
- `RotatingFileHandler` caps log size and keeps three old files.
- The top-level `try/except` writes the traceback to the log with `log.exception`. A scheduled job has nobody watching, so the log is the only witness.
- `--every-seconds` and `--runs` give a Python-only repeat. It lasts only while the program runs.

**Sample input:** `sample_data/inbox/expenses_latest.csv` (the expense data from Project 9).

**Sample output** (real output from my test run, Windows 11, Python 3.14.4, 2026-11-07):

First run:

```
Wrote report_2026-11-07.md from expenses_latest.csv (5
```

```
categories)
```

Second run the same day:

```
Report report_2026-11-07.md already exists: nothing to do
```

Two runs one second apart, new output folder:

```
Wrote report_2026-11-07.md from expenses_latest.csv (5
categories)
Report report_2026-11-07.md already exists: nothing to do
```

**How to run it** (from the `source` folder):

```
py project25_scheduled_report.py sample_data/inbox reports
py project25_scheduled_report.py sample_data/inbox reports
py project25_scheduled_report.py sample_data/inbox reports_b --
every-seconds 1 --runs 2 --log job_b.log
```

**Common errors and fixes:**

- **No CSV files in .** Check the inbox path in the scheduler's command; the working folder may differ.
- **Nothing happens when the scheduler runs.** Use absolute paths for python, the script and all folders.

**Safe-use notes:** The scheduled setup itself (Task Scheduler, cron, launchd) was not tested for this book. Test it with a harmless script first, and give the job only the access it needs.

**Optional challenge:** Exit with code 3 when the newest CSV is older than 7 days, so a scheduler can alert you.

CHAPTER 12

# Debugging, security and next steps

## Reading a traceback

Read from the bottom. The last line names the error and message. The lines above show the chain of calls; the one nearest the bottom in your own file is usually where to look.

```
Traceback (most recent call last):
  File "demo.py", line 3, in <module>
    total = int(price) * 2
ValueError: invalid literal for int() with base 10: '4.5'
```

Here the message says `int` cannot convert the text `4.5`. The fix is `float(price)`.

## Three debugging tools

- **print:** print the variable just before the line that fails. Remove the prints afterwards.
- **logging:** like print but with levels and a file; good for scheduled jobs.
- **pdb:** the built-in debugger. Add `breakpoint()` before the failing line and run the script. At the `(Pdb)` prompt, `p variable` prints a value, `n` runs the next line, `c` continues, `q` quits.

## Safe file handling checklist

- [ ] Work on a copy of the data while testing.
- [ ] Take a backup (Project 4) before any move, rename or in-place edit.
- [ ] Offer a dry run, and make it the default.
- [ ] Refuse to overwrite; write to a new name.
- [ ] Never delete automatically. Report instead.
- [ ] Validate paths and inputs, and print a clear error.
- [ ] Log what the script did.
- [ ] Count inputs and outputs and compare.

## Secrets

Do not write passwords, API keys or tokens in script files or in the folder you back up or share. Use environment variables or your operating system's credential store. If a secret was ever saved in a file you shared, treat it as exposed and replace it at its source. Email scripts: prepare drafts (Project 23) rather than storing a mail password.

## Dependency hygiene

- Install packages only from the official package index, and check the name for typos; lookalike names exist.
- Keep a `requirements.txt`, and use a virtual environment per project.
- Update on purpose, then rerun your tests. `py -m pip list --outdated` shows what is old.
- Fewer dependencies mean fewer problems. Prefer the standard library when it does the job.

## Next steps

Pick one task from your own week and automate it with the pattern from Chapter 1. Then explore: reading EXIF data with Pillow, `python-docx` for Word, OCR for scans, `pytest` for tests, Git to track versions of

your scripts, and the official Python tutorial at docs.python.org.

# Conclusion

Automation is a habit more than a skill: notice a repeated task, write the steps, test on a copy, run safely. You now have 25 working examples of that cycle and a checklist to apply to new ones.

An action list for this week:

- [ ] Set up Python and a virtual environment (Chapter 2).
- [ ] Run Projects 1 and 5 on sample data, then on a copy of a real folder.
- [ ] Choose one monthly task and adapt Project 9, 13 or 22 to your own data.
- [ ] Add a dry run and a log to every script you write.
- [ ] Back up your scripts and data in two places.

# Appendix A. Python cheat sheet

| Task | Code |
|---|---|
| Join a path | `Path("a") / "b.txt"` |
| List files | `Path(".").glob("*.csv")` |
| File name parts | `p.name`, `p.stem`, `p.suffix` |
| Read text | `p.read_text(encoding="utf-8")` |
| Write text | `p.write_text(s, encoding="utf-8")` |
| Does it exist | `p.exists()` |
| Copy / move | `shutil.copy2(a, b)`, `shutil.move(a, b)` |
| Strip and split | `s.strip()`, `s.split(",")` |
| Count items | `Counter(items).most_common(5)` |
| Dictionary default | `d.get(key, 0)` |
| Sort by value | `sorted(d.items(), key=lambda x: x[1])` |
| Parse a date | `datetime.strptime(s, "%Y-%m-%d")` |
| Format a date | `dt.strftime("%Y%m%d_%H%M%S")` |
| Read CSV | `csv.DictReader(fh)` |
| JSON to Python | `json.loads(text)` |
| Environment variable | `os.environ.get("NAME")` |
| Format money | `f"{x:,.2f}"` |
| Run main only directly | `if __name__ == "__main__":` |

# Appendix B. Common error reference

| Error | Usual cause | Fix |
|---|---|---|
| `SyntaxError` | Missing colon, bracket or quote | Check the line shown and the one above it |
| `IndentationError` | Uneven spaces | Use four spaces everywhere |
| `NameError` | Misspelt or undefined name | Check spelling and order |
| `ModuleNotFoundError` | Package not installed in this Python | `py -m pip install name` in the active environment |
| `FileNotFoundError` | Wrong path or folder | Print `Path.cwd()`; use absolute paths |
| `PermissionError` | File open elsewhere or protected | Close the program; check permissions |
| `UnicodeDecodeError` | Wrong text encoding | Try `encoding="utf-8-sig"` or `"cp1252"` |
| `KeyError` | Missing dictionary key or column | Check the header; use `.get` |
| `ValueError` | Bad conversion such as `int("abc")` | Validate input first |
| `TypeError` | Wrong kind of value | Convert with `int()`, `str()` |

| Error | Usual cause | Fix |
| --- | --- | --- |
| `IndexError` | List position out of range | Check the length first |
| `ZeroDivisionError` | Dividing by zero | Test the divisor |
| `python is not recognized` | Python not on PATH | Use `py`, or reinstall with PATH ticked |
| `pip` installs to the wrong Python | Plain `pip` used | Use `py -m pip` |

# Appendix C. Glossary

**API:** a defined way for programs to request data or actions from each other.

**argument:** a value given to a script on the command line, or to a function.

**backup:** a separate copy of data kept so you can recover from mistakes.

**CSV:** plain text table with values separated by commas.

**dry run:** a run that shows what would happen without doing it.

**encoding:** the rule that maps characters to bytes; UTF-8 is the common choice.

**environment variable:** a named setting held by the operating system, often used for secrets.

**exit code:** the number a program returns; 0 means success.

**hash:** a fixed-length fingerprint of data, such as SHA-256; the same content always gives the same hash.

**idempotent:** safe to run twice, because the second run changes nothing.

**JSON:** a text format for lists and key-value data, common in APIs.

**OCR:** optical character recognition, turning images of text into text.

**package:** code written by others that you install with pip.

**regular expression:** a pattern for matching text.

**traceback:** the report Python prints when an error stops a script.

**virtual environment:** a private folder of packages for one project.

# Appendix D. Project checklist

Use this before you run any script on real data.

- [ ] I know what the script changes and where.
- [ ] I ran it on sample data first.
- [ ] I have a backup, or the script only reads.
- [ ] I ran the dry run and read all of it.
- [ ] The output name does not exist yet.
- [ ] No password, key or personal data is written in the code.
- [ ] I know how to undo the result.
- [ ] I compared counts before and after.
- [ ] I saved the script and a note of how I ran it.

# Appendix E. 30-day learning plan

About 30 to 45 minutes a day. Adjust freely; skipping a day is fine.

- [ ] Day 1: Install Python; run `py --version`.
- [ ] Day 2: Create and activate a virtual environment.
- [ ] Day 3: Install openpyxl and pypdf; run `make_sample_data.py`.
- [ ] Day 4: Chapter 3, variables and strings.
- [ ] Day 5: Lists, dictionaries, loops.
- [ ] Day 6: Functions and exceptions.
- [ ] Day 7: Review; write a script that prints the files in a folder.
- [ ] Day 8: Project 5, folder size report.
- [ ] Day 9: Read Project 5's code line by line; change the output.
- [ ] Day 10: Project 1, bulk rename, dry run only.
- [ ] Day 11: Project 1, apply on a copy.
- [ ] Day 12: Project 2, organise by type.
- [ ] Day 13: Project 3, duplicates.
- [ ] Day 14: Project 4, backup; restore one file by hand.
- [ ] Day 15: Review week two; list tasks from your own work.
- [ ] Day 16: Project 7, CSV cleanup.
- [ ] Day 17: Project 9, expense summary on your own data.
- [ ] Day 18: Project 11, validation rules for your file.
- [ ] Day 19: Project 13, Excel report.
- [ ] Day 20: Project 14, merge workbooks.
- [ ] Day 21: Review; adapt one project to your data.
- [ ] Day 22: Project 15, PDF text extraction on a PDF of yours.
- [ ] Day 23: Projects 16 and 17, merge and split.

- [ ] Day 24: Projects 18 and 19, text tools.
- [ ] Day 25: Project 21, log summary.
- [ ] Day 26: Project 22, monthly report.
- [ ] Day 27: Project 24, API; read the terms of one API.
- [ ] Day 28: Project 25, logging and exit codes.
- [ ] Day 29: Set up a schedule for a harmless script.
- [ ] Day 30: Choose your first real automation and write its plan.

# Appendix F. References and further reading

The sources below were opened on 2026-11-07 ("accessed 2026-11-07"). A page that failed to open is noted.

- https://www.python.org/downloads/ - current releases and downloads (read in the preparation research on 2026-11-07, which noted newer series than 3.14; the book names no patch number except the one I ran).
- https://docs.python.org/3/library/venv.html - virtual environment creation and activation commands. Accessed 2026-11-07.
- https://pypdf.readthedocs.io/en/stable/user/extract-text.html - text extraction limits, scanned PDFs and OCR. Accessed 2026-11-07.
- https://jsonplaceholder.typicode.com/ - the free test API used in Project 24. Accessed 2026-11-07.
- https://www.rfc-editor.org/rfc/rfc9309 - Robots Exclusion Protocol; robots.txt is not access authorisation. Accessed 2026-11-07.
- https://learn.microsoft.com/en-us/windows/win32/taskschd/task-scheduler-start-page - what Windows Task Scheduler does and its triggers. Accessed 2026-11-07.
- https://openpyxl.readthedocs.io/en/stable/simple_usage.html - returned HTTP 404 when accessed 2026-11-07, so it is not used as a source.

- Competitor Gumroad pages (for example numbpilled.gumroad.com/l/pythonninja, vigneshselvaraj.gumroad.com/l/python-automation-playbook) - opened 2026-11-07; most showed only a title, which is why the market comparison in the Introduction is limited.

Full notes are in `research_notes.md` in the book folder.

## Bonus download

`Python_Automation_Project_Source.zip` contains the `source` folder: all 25 scripts, `make_sample_data.py`, the sample data, `requirements.txt`, `README.md` and `test_log.md`. It is an optional bonus; the book is complete without it. Unzip it, create a virtual environment inside, install the requirements, and run `py make_sample_data.py` to recreate the sample files at any time.